

Implementation of Digital Om chanting on Raspberry Pi using Generated Dataset

A Project Report

Submitted by

LUBHAWAN VERMA (22EBKCS067)

PRIYANSHU (22EBKDS045)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

at



B.K. BIRLA INSTITUTE OF ENGINEERING & TECHNOLOGY

PILANI, RAJASTHAN (INDIA) - 333031

(AFFILIATED TO RAJASTHAN TECHNICAL UNIVERSITY, KOTA, RAJASTHAN (INDIA))

APRIL 2026

CANDIDATE’S DECLARATION

I hereby declare that the work presented in this project entitled “**Implementation of Digital Om chanting on Raspberry Pi using Generated Dataset**”, submitted in partial fulfillment for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**, at **B.K. Birla Institute of Engineering and Technology, Pilani**, is a record of my own work carried out under the guidance of **Mr. Gauttam Jangir**, Department of Computer Science and Engineering.

I further declare that this project work has not been submitted by me anywhere else for the award of any other degree, diploma, or any other similar title.

Signature of the Student
(Lubhawan Verma & Priyanshu)

Place:

Date:

CERTIFICATE

This is to certify that the project titled “**Implementation of Digital Om chanting on Raspberry Pi using Generated Dataset**” is the bonafide work carried out by **Lubhawan Verma (B.Tech – Computer Science and Engineering)** and **Priyanshi (B.Tech – Computer Science and Engineering – Data Science)** at **B.K. Birla Institute of Engineering and Technology, Pilani**, affiliated to **Bikaner Technical University, Bikaner, Rajasthan (India)**, during the academic year **2025-2026**, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology**.

It is further certified that this project work has not formed the basis for the award of any other degree, diploma, or any other similar title.

Signature of the Guide
(Mr. Gauttam Jangir)
Assistant Prof. BKBIET
Department of Computer Science

Place:

Date:

INDEX

1. Introduction
 - 1.1 Background
 - 1.2 Problem Statement
 - 1.3 Purpose of the System
2. System Overview
 - 2.1 Concept of Meditation Tracking
 - 2.2 How the System Works
 - 2.3 Raspberry Pi Implementation
 - 2.3.1 Introduction of Raspberry Pi Board
 - 2.3.2 Hardware Components
 - 2.3.3 Software Environment
 - 2.3.4 System Integration
 - 2.3.5 Real Time Processing and Optimization
 - 2.3.6 Working of the System on Raspberry Pi
 - 2.3.7 Advantages of Raspberry Pi Implementation
 - 2.3.8 Limitations and Challenges
 - 2.3.9 Possible Improvements
 - 2.3.10 Real-World Applications
 - 2.3.11 Conclusion of Implementation
3. Objectives of the Project
4. Key Features
5. Detailed Module Description
 - 5.1 Face Detector Module
 - 5.2 Session Tracker Module
 - 5.3 Audio Manager Module
 - 5.4 Face Database Module
 - 5.5 Streamlit Interface (Main Application)
6. Feature Breakdown

- 6.1 Eye Detection
- 6.2 Neutrality & Emotion Detection
- 6.3 Audio Feedback System
- 6.4 Expression Stability Analysis
- 6.5 Real-Time Visualization
- 6.6 Session Historical Storage
- 7. Algorithms Used
 - 7.1 Eye Aspect Ratio (EAR) Algorithm
 - 7.2 Mouth Ratio & Smile Detection
 - 7.3 Facial Emotion Detection
 - 7.4 Om Sound Triggering Logic
- 8. User Interface Design
 - 8.1 Introduction to User Interface Design
 - 8.2 Design Philosophy and Human-Centered Approach
 - 8.3 Overall Layout of the Interface
 - 8.4 Sidebar Controls
 - 8.5 Main Display Panel
 - 8.6 Real-Time Metrics Display
 - 8.7 Audio Feedback Indicator
 - 8.8 Session Information Display
 - 8.9 Real-Time Data Handling and Synchronization
 - 8.10 User Interaction Flow
 - 8.11 Error Handling and User Guidance
 - 8.12 Performance Optimization in UI
 - 8.13 Accessibility and Usability Considerations
 - 8.14 Limitations and Future Improvements
 - 8.15 Application Interface
- 9. Performance Evaluation
- 10. Data Storage & Analytics
- 11. Configuration & Customization Options

12. Error Handling & System Robustness
13. Source Code
14. Deep Learning Model for Neutral Detection
 - 14.1 Dataset Description
 - 14.2 Data Preprocessing
 - 14.3 Model Architecture
 - 14.4 Training Procedure
 - 14.5 Evaluation & Results
 - 14.6 Exporting and Using the Model in the Project
15. Installation & Setup Guide
16. Troubleshooting Guide
17. Appendices
 - 17.1 Dataset References
 - 17.2 Dependency Versions
 - 17.3 File Structure
18. Mathematical Formulation of Neutrality Score
19. Interface
20. Model Training Challenges
21. Real-World Scenarios and Case Studies
22. Future AI Model Improvements
23. Conclusion
24. Reference

CHAPTER - 1

Introduction

In recent years, there has been a growing interest in integrating technology with mental wellness practices. With increasing levels of stress, anxiety, and distractions in modern life, meditation has emerged as a powerful tool for improving mental clarity, emotional stability, and overall well-being. However, while meditation is simple in concept, maintaining a consistent and focused meditative state is often challenging for individuals, especially beginners.

Most existing digital meditation solutions, such as mobile applications and guided audio sessions, primarily focus on providing instructions and timing assistance. Although these tools are helpful, they lack the capability to monitor the actual state of the user during meditation. They do not consider whether the user's eyes are closed, whether the face is relaxed, or whether the user is emotionally stable. As a result, these systems fail to provide objective and real-time feedback, which is essential for improving the quality of meditation practice.

Advancements in Artificial Intelligence, Computer Vision, and Embedded Systems have opened new possibilities for addressing these limitations. Modern computer vision techniques enable the detection of facial landmarks, eye movements, and subtle expression changes using a simple camera. Additionally, machine learning and deep learning models can analyze emotional states and classify facial expressions with high accuracy. These technologies provide an opportunity to create intelligent systems that can observe, analyze, and respond to human behavior in real time.

This project leverages these advancements to develop a smart meditation assistant that can automatically monitor a user's facial expressions and provide immediate feedback. The system uses facial recognition and landmark detection techniques to evaluate key indicators of meditation, such as eye closure, facial stillness, and emotional neutrality. Based on these observations, the system determines whether the user is in a proper meditative state.

A unique feature of this system is its real-time audio feedback mechanism. When the user maintains a calm and neutral expression with closed eyes, the system plays a soothing "Om" chanting sound, reinforcing the meditative state. If any disturbance is detected, such as opening of eyes or changes in facial expressions, the audio feedback stops instantly. This continuous interaction helps users stay focused and become more aware of their meditation performance.

Furthermore, the system is implemented on a Raspberry Pi platform, making it portable, energy-efficient, and cost-effective. This allows the solution to function as a standalone device without the need for high-end computing resources. The portability of the system makes it suitable for personal use, wellness centers, and even clinical or therapeutic environments.

Overall, this project represents a significant step toward combining traditional meditation practices with modern intelligent technologies. It aims to provide a more interactive, adaptive, and effective meditation experience by introducing automation, real-time feedback, and performance tracking.

1.1 Background

Meditation is widely recognized as an effective technique for improving mental clarity, emotional stability, and overall well-being. It requires a high level of concentration, stillness, and control over both physical and emotional responses. However, in practical scenarios, especially for beginners, maintaining this level of consistency is difficult. Users often lose focus without realizing it, either by opening their eyes, making subtle facial movements, or reacting to internal thoughts and emotions.

Traditional meditation practices depend heavily on self-awareness, which may not always be accurate. Even modern meditation applications primarily provide guided audio or timers but lack the capability to monitor the user's physical and emotional state in real time. As a result, users do not receive objective feedback about whether they are truly maintaining a meditative state.

With the rapid advancement in Artificial Intelligence and Computer Vision, it has become possible to analyze facial expressions, eye movements, and emotional cues using camera-based systems. These technologies enable the development of intelligent systems that can monitor user behavior continuously and provide real-time feedback. This creates a strong foundation for building a smart meditation assistant that can enhance traditional meditation practices with measurable and automated monitoring.

1.2 Problem Statement

Despite the growing popularity of meditation, existing solutions suffer from significant limitations. Most meditation tools rely on self-reporting or subjective judgment, where users evaluate their own performance. This approach is inherently unreliable, as individuals may not accurately perceive their level of focus, emotional stability, or physical stillness during meditation.

Furthermore, current applications lack the ability to detect real-time physiological indicators such as eye closure, facial neutrality, and expression stability. These indicators are crucial in determining whether a person is truly in a calm and focused state. Without objective monitoring, users may unknowingly break their concentration, reducing the effectiveness of meditation sessions.

Another key limitation is the absence of instant feedback mechanisms. Users are not alerted when they lose focus, making it difficult to correct their behavior during the session itself. This gap highlights the need for a system that can automatically observe facial features, analyze emotional states, and provide immediate feedback.

Therefore, there is a clear requirement for a simple, cost-effective, and real-time system that uses computer vision techniques to detect facial neutrality and eye closure, enabling accurate and continuous guidance during meditation.

1.3 Purpose of the System

The primary purpose of this system is to assist individuals in achieving and maintaining a deeper state of

meditation through real-time monitoring and feedback. The system tracks key indicators such as eye closure, facial neutrality, and emotional stability using computer vision techniques.

By continuously analyzing these parameters, the system provides immediate audio feedback in the form of “Om” chanting when the user is in a calm and focused state. If any disturbance is detected, such as opening of eyes or changes in facial expression, the audio feedback is stopped instantly. This real-time response helps users become aware of distractions and correct their behavior during the meditation session.

In addition to real-time guidance, the system also records session data, allowing users to evaluate their performance over time. This promotes self-improvement by providing measurable insights into meditation consistency and emotional control.

The system is designed to be portable and efficient by implementing it on a Raspberry Pi, making it suitable for personal use as well as wellness centers. Overall, the purpose of this project is to combine traditional meditation practices with modern technology to create a structured, interactive, and data-driven meditation experience.

CHAPTER – 2

System Overview

The proposed system is designed as an intelligent, real-time meditation monitoring solution that integrates computer vision, machine learning, and embedded system technologies. Its primary objective is to observe and evaluate the user's meditative state by analyzing facial expressions and eye behavior, and to provide immediate feedback that helps maintain focus and stillness during meditation.

Unlike traditional meditation techniques, which rely heavily on self-awareness and subjective judgment, this system introduces an automated and data-driven approach. It continuously monitors the user through a camera and processes visual data to identify key indicators of meditation, such as eye closure, facial neutrality, and emotional stability. By converting these physical indicators into measurable parameters, the system enables objective evaluation of meditation quality.

The overall architecture of the system is designed to operate in a structured and modular manner. It begins with capturing real-time video input, followed by processing the data to extract meaningful features. These features are then analyzed using predefined algorithms and decision-making logic to determine whether the user is in a valid meditative state. Based on this evaluation, the system generates appropriate feedback in the form of audio signals.

A key aspect of the system is its ability to function in real time. Each video frame is processed instantly, ensuring that the system can detect even small changes in the user's facial expressions or eye movements. This real-time capability allows the system to respond immediately, providing continuous guidance throughout the meditation session. Such responsiveness is essential for maintaining user engagement and improving the effectiveness of meditation practice.

Another important feature of the system is its modular design. The system is divided into multiple functional components, including face detection, feature extraction, emotion analysis, decision-making, audio management, and data storage. Each module performs a specific task, and all modules work together in a synchronized pipeline. This modular approach improves system flexibility, making it easier to modify, upgrade, or extend individual components without affecting the overall functionality.

The system also incorporates a feedback mechanism that plays a crucial role in guiding the user. When the system detects that the user is in a calm and stable meditative state, it activates a soothing “Om” chanting sound. This audio feedback reinforces the user's focus and helps maintain a deeper level of meditation. Conversely, when the system detects a disturbance, such as eye opening or facial movement, the audio is stopped immediately. This instant feedback allows users to become aware of their distractions and correct them in real time.

In addition to real-time monitoring and feedback, the system also supports session tracking and data storage. It records various parameters, such as session duration, neutrality scores, and eye state changes, which can be used for performance evaluation. This data-driven approach enables users to analyze their progress over time and improve their meditation practices in a structured manner.

Furthermore, the system is designed to be deployed on a Raspberry Pi platform, making it portable, cost-effective, and suitable for real-world usage. The embedded implementation ensures that the system can

operate as a standalone device without requiring a full computer setup. This enhances its usability and makes it accessible to a wide range of users.

Overall, the system overview highlights how different components and technologies are integrated to create a comprehensive meditation tracking solution. The following sections provide a detailed explanation of the core concepts, working mechanism, and implementation of the system.

2.1 Concept of Meditation Tracking

Meditation tracking refers to the process of observing and evaluating a user's meditative state using measurable physical and behavioral indicators. Traditionally, meditation has been considered a subjective practice, where individuals rely on their own perception to determine whether they are focused or distracted. However, this approach lacks accuracy, as users often become unaware of subtle distractions such as opening their eyes, making facial movements, or experiencing emotional fluctuations.

The concept of meditation tracking in this system is based on the principle that internal mental states are reflected through external physical expressions, particularly facial features and eye behavior. When a person is in a calm and focused meditative state, their eyes remain closed, facial muscles are relaxed, and there is minimal variation in expression. On the other hand, distractions or emotional disturbances often lead to noticeable changes such as eye opening, smiling, frowning, or muscle tension.

To capture and analyze these indicators, the system uses computer vision techniques to monitor the user's face in real time. A camera continuously captures video frames, and facial landmark detection algorithms are applied to identify key regions such as the eyes, lips, and eyebrows. These landmarks provide precise spatial information that can be used to quantify facial movements and expressions.

One of the primary parameters used in meditation tracking is eye closure detection, which is calculated using the Eye Aspect Ratio (EAR). This metric helps determine whether the user's eyes are open or closed at any given moment. Consistent eye closure is considered an important indicator of focus and internal awareness during meditation.

Another critical parameter is facial neutrality, which represents the absence of strong emotional expressions. The system evaluates neutrality by analyzing features such as mouth shape, lip curvature, and eyebrow positioning. These features are combined to generate a neutrality score, which ranges from 0 to 1. A higher score indicates a calm and stable facial state, while a lower score suggests emotional activity or distraction.

In addition to these features, the system may optionally use deep learning-based emotion detection models to classify facial expressions into categories such as happy, sad, angry, or neutral. This enhances the accuracy of meditation tracking by providing deeper insight into the user's emotional condition.

To ensure stability and avoid sudden fluctuations caused by noise or minor movements, the system applies smoothing techniques by averaging values over multiple frames. This creates a more reliable representation of the user's state over time rather than reacting to instant changes.

By combining these parameters, the system transforms meditation from a subjective experience into a measurable and data-driven process. It continuously evaluates whether the user meets the conditions of a proper meditative state, such as closed eyes and high facial neutrality.

Overall, the concept of meditation tracking in this project provides a scientific and structured approach to understanding meditation quality. It enables real-time monitoring, objective feedback, and performance evaluation, helping users improve their focus, maintain stillness, and achieve deeper levels of meditation more effectively

2.2 How the System Works

The proposed system operates as a real-time intelligent pipeline that continuously monitors the user's facial expressions and eye behavior to determine their meditation state. It integrates computer vision techniques, feature extraction methods, and decision-making logic to provide immediate feedback based on the user's condition.

The working of the system begins with **video acquisition**, where a webcam or Raspberry Pi camera module captures live video frames of the user. These frames are processed sequentially to ensure continuous monitoring. The system is designed to operate in real time, meaning each frame is analyzed instantly without noticeable delay.

Once a frame is captured, the system performs **face detection** using the MediaPipe framework. This step ensures that a valid human face is present in the frame. If no face is detected, the system ignores the frame and waits for valid input. When a face is detected, the system proceeds to extract detailed facial landmarks.

In the next step, **facial landmark detection** is applied to identify key points on the face, including the eyes, lips, and eyebrows. MediaPipe Face Mesh provides a dense set of landmarks that accurately represent facial geometry. These landmarks are essential for calculating various features that indicate the user's meditative state.

After landmark extraction, the system performs **feature analysis**. One of the primary features is the **Eye Aspect Ratio (EAR)**, which is used to determine whether the eyes are open or closed. A threshold value is defined, and if the EAR falls below this threshold, the system classifies the eyes as closed. This is an important condition for meditation detection.

In addition to eye analysis, the system evaluates **facial expressions** by calculating parameters such as mouth ratio, lip curvature, and eyebrow movement. These features help detect whether the face is neutral or showing signs of emotion. Optionally, a deep learning-based emotion detection model (such as DeepFace) can be used to classify emotions more accurately.

The extracted features are then combined in the **neutrality computation stage**, where a neutrality score is calculated. This score represents the level of calmness and stability of the user's facial expression. To improve reliability, the system applies smoothing techniques by averaging values over multiple frames, reducing the impact of sudden fluctuations or noise.

Once the neutrality score and eye state are determined, the system enters the **decision-making phase**. In this stage, predefined conditions are evaluated:

- If the eyes are closed and the neutrality score is above a certain threshold, the system considers the user to be in a valid meditative state.
- If these conditions are not met, the system detects a disturbance.

Based on this decision, the system activates the **audio feedback mechanism**. When the user maintains a proper meditative state, a soothing “Om” chanting sound is played continuously. If the user opens their eyes or changes facial expressions, the sound stops immediately. This real-time feedback helps the user maintain focus and quickly correct any distractions.

Simultaneously, the system performs **session tracking and data logging**. It records important information such as session duration, neutrality scores over time, and the frequency of eye closure. This data is stored in a lightweight format (such as JSON) and can be used for later analysis and performance evaluation.

Overall, the system works as a continuous loop of capturing, analyzing, deciding, and responding. This real-time interaction between the user and the system creates an adaptive meditation environment, where feedback is instant and based on measurable parameters. As a result, users can improve their meditation practice in a structured and effective manner.

2.3 Raspberry Pi Implementation

In addition to running on a standard computer, the proposed system is designed to operate on an embedded platform such as the Raspberry Pi. This makes the solution portable, cost-effective, and suitable for real-world deployment

2.3.1 Introduction of Raspberry Pi Board

The implementation of the meditation tracking system on a Raspberry Pi platform represents a significant step toward converting a conceptual software model into a fully functional embedded system. Raspberry Pi is a versatile single-board computer that has gained popularity due to its compact size, affordability, and capability to handle real-time applications. It provides an ideal platform for integrating computer vision, machine learning, and interactive feedback systems into a single device.

In the context of this project, Raspberry Pi serves as the core computational unit that executes all major operations, including video acquisition, facial analysis, feature extraction, decision-making, and audio feedback generation. Unlike traditional implementations that rely on desktop computers or high-performance GPUs, this system is designed to function independently on low-power hardware. This makes it highly suitable for real-world usage, where portability and efficiency are critical.

The embedded nature of the system ensures that it can operate continuously without requiring additional infrastructure. This enables users to perform meditation sessions in a more natural and distraction-free environment. Furthermore, the use of Raspberry Pi aligns with the goal of creating a cost-effective and accessible solution that can be deployed at scale in wellness centers, therapy clinics, and personal environments.

2.3.2 Hardware Components

The hardware architecture of the system is intentionally kept simple to ensure ease of implementation while maintaining functionality and performance.

◇ Raspberry Pi Board

The Raspberry Pi 5 Model B is recommended due to its improved CPU performance, increased RAM capacity, and better support for multimedia processing. It features a quad-core processor that is capable of handling real-time video processing tasks to a reasonable extent. The board acts as the central controller that coordinates all system components and executes the main application.



Fig 2.1

◇ Camera Module

A camera is a critical component of the system, as it provides the visual input required for facial analysis. The system supports both USB webcams and the official Raspberry Pi Camera Module. The Pi Camera, connected via the CSI interface, offers better integration and lower latency, while USB webcams provide flexibility and ease of use. A resolution of at least 720p is recommended to ensure accurate detection of facial landmarks and expressions.



Fig :2.2

◇ Audio Output Device

The system uses speakers or headphones to provide real-time audio feedback. The “Om” chanting sound acts as a reinforcement signal when the user maintains a proper meditative state. Audio devices can be connected through the 3.5mm audio jack or USB ports, depending on the available hardware configuration.



Fig:2.3

◇ Storage (MicroSD Card)

The microSD card serves as the primary storage medium for the Raspberry Pi. It contains the operating system, application code, required libraries, and recorded session data. A minimum capacity of 32GB is recommended to ensure smooth performance and sufficient storage for logs and datasets.



Fig:2.4

◇ Power Supply

A stable 5V/3A power supply is required to ensure consistent performance. Insufficient power can lead to system instability, especially during high processing loads.

◇ Optional Components

Additional components such as heat sinks, cooling fans, HDMI display, keyboard, and mouse can be used for setup, monitoring, and thermal management. Cooling is particularly important for maintaining performance during long meditation sessions.

2.3.3 Software Environment

The software stack of the system is carefully chosen to balance performance, flexibility, and ease of development.

♦ Operating System

The system runs on Raspberry Pi OS, which is a Debian-based Linux distribution optimized for the Raspberry Pi hardware. It provides a stable and lightweight environment with support for essential drivers and libraries.

♦ Programming Language

Python 3 is used as the primary programming language due to its simplicity, readability, and extensive ecosystem of libraries for computer vision and machine learning. Python also allows rapid prototyping and easy integration of different modules.

♦ Required Libraries

- OpenCV → Used for capturing video streams, processing images, and performing basic computer vision operations such as resizing, filtering, and frame manipulation.
- MediaPipe → Provides highly efficient facial landmark detection, enabling the system to track key facial points with high accuracy and speed.
- NumPy → Used for mathematical computations, array operations, and implementation of algorithms such as EAR and neutrality calculations.
- Streamlit → Used to build a simple and interactive graphical user interface that displays real-time metrics such as eye state, neutrality score, and session duration.
- Pygame / Pydub → Used for handling audio playback, including playing and stopping the “Om” chanting sound based on system decisions.
- DeepFace (optional) → Used for advanced emotion detection using deep learning models. This module can be enabled or disabled depending on system performance requirements.

These libraries work together to enable real-time analysis and interaction.

2.3.4 System Integration

System integration involves combining all hardware and software components into a unified working system. The camera is configured using Video4Linux (V4L2) drivers, which allow efficient access to video streams. Audio output is configured using the ALSA (Advanced Linux Sound Architecture) system, ensuring compatibility with Linux-based environments.

The Python environment is set up with all necessary dependencies installed through pip. Once configured, the application is executed using Streamlit, which launches a web-based interface accessible through a browser. The system begins capturing video, analyzing frames, and providing feedback in real time.

All modules are interconnected in a pipeline manner. The face detection module feeds data to the feature extraction module, which in turn passes processed data to the decision-making module. The output module then generates audio feedback, while the storage module logs session data. This seamless integration ensures smooth and continuous operation.

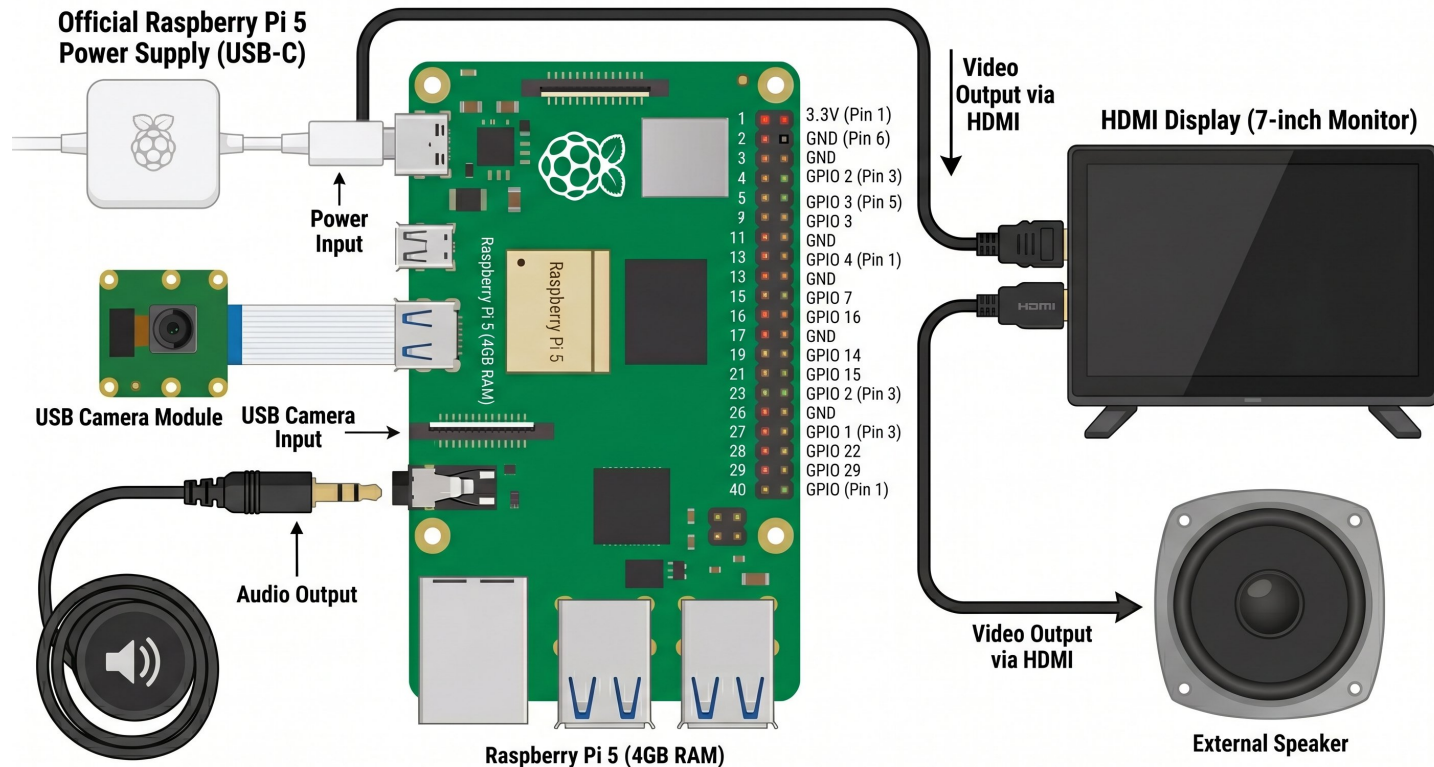


Fig: Circuit Design

2.3.5 Real-Time Processing and Optimization

Since Raspberry Pi has limited computational resources, optimization is essential for maintaining real-time performance.

♦ Frame Optimization

Video resolution is reduced, and frames may be processed selectively to reduce load.

♦ Lightweight Processing

Heuristic methods are used instead of heavy deep learning models when possible. DeepFace is kept optional.

♦ Backend Optimization

- V4L2 is used for efficient video capture
- ALSA ensures smooth audio output

◇ Memory Management

JSON-based storage is used instead of databases to reduce memory usage and improve speed.

◇ CPU Optimization

Efficient coding practices and controlled processing loops are used to minimize CPU load.

2.3.6 Working of the System on Raspberry Pi

The system operates in a continuous loop, performing the following steps:

- Capture video frames using the camera
- Detect the presence of a face
- Extract facial landmarks
- Calculate features such as EAR and mouth ratio
- Compute neutrality score
- Apply decision logic
- Trigger audio feedback (play/stop Om sound)
- Store session data

This pipeline ensures real-time interaction and feedback. Once deployed, the system operates in a continuous loop. The camera captures video frames, which are then processed to detect faces and extract facial landmarks. Key features such as eye aspect ratio and facial expression metrics are calculated. These features are used to compute a neutrality score, which represents the user's level of calmness.

The system then evaluates predefined conditions to determine whether the user is in a valid meditative state. If the conditions are satisfied, the system plays the “Om” chanting sound; otherwise, the sound is stopped. Simultaneously, session data is recorded for future analysis. This loop continues throughout the meditation session, ensuring continuous monitoring and feedback.

2.3.7 Advantages of Raspberry Pi Implementation

The use of Raspberry Pi provides several benefits:

- **Portability:** Compact and easy to carry
- **Cost-effectiveness:** Affordable compared to traditional systems
- **Low power consumption:** Suitable for long-term usage
- **Standalone operation:** No need for external computer
- **Scalability:** Can be integrated with IoT or cloud systems

2.3.8 Limitations and Challenges

Despite its advantages, some challenges exist:

- Limited processing power
- Reduced performance with deep learning models
- Sensitivity to lighting conditions
- Heating during continuous operation

2.3.9 Possible Improvements

The system can be enhanced through:

- Use of hardware accelerators (e.g., Coral TPU)
- Improved lighting setup
- Advanced optimized models
- Cloud-based analytics
- Mobile application integration

2.3.10 Real-World Applications

The Raspberry Pi-based system can be applied in various domains:

- Personal meditation assistance
- Yoga and wellness centers
- Mental health therapy
- Corporate wellness programs

2.3.11 Conclusion of Implementation

The Raspberry Pi implementation successfully converts the meditation tracking system into a compact, efficient, and deployable embedded solution. By integrating computer vision, real-time processing, and audio feedback within a single device, the system provides a practical and scalable approach to enhancing meditation practices. Its portability, affordability, and standalone capability make it highly suitable for real-world applications.

CHAPTER – 3

Objectives of the Project

The primary objective of this project is to develop an intelligent and real-time meditation tracking system that enhances the overall effectiveness of meditation practices by integrating computer vision, machine learning, and embedded system technologies. The system is designed to provide objective and measurable feedback based on the user's physical and emotional state, thereby transforming meditation from a subjective experience into a structured and data-driven process. Unlike traditional meditation approaches that rely on self-awareness, this system introduces automation and real-time interaction, enabling users to understand and improve their meditation performance more effectively.

A key objective of the project is to design a system capable of continuously monitoring the user in real time. The system captures live video input using a camera and processes each frame to analyze facial expressions and eye behavior. This real-time capability ensures that even the smallest changes in the user's state are detected instantly. By providing immediate feedback, the system helps users remain aware of their actions and maintain focus throughout the meditation session. This continuous monitoring plays a crucial role in improving the quality and consistency of meditation practice.

Another important objective is to accurately detect eye closure, which is a fundamental indicator of meditation. The system uses facial landmark detection techniques to identify key points around the eyes and calculate parameters such as the Eye Aspect Ratio (EAR). This allows the system to determine whether the user's eyes are open or closed at any given moment. Ensuring accurate eye detection helps in identifying distractions and maintaining proper meditation posture, as frequent eye opening often indicates loss of focus.

In addition to eye detection, the system aims to analyze facial expressions to determine emotional stability and neutrality. A calm and neutral facial expression is a key characteristic of effective meditation. The system evaluates features such as mouth shape, lip curvature, and eyebrow position to detect any changes in expression. These features are used to determine whether the user is maintaining a relaxed and stable state or experiencing emotional fluctuations. The system may also incorporate deep learning-based emotion detection models to enhance accuracy and provide deeper insights into the user's emotional condition.

Another major objective is to compute a neutrality score that represents the user's level of calmness and stability during meditation. This score is derived by combining multiple facial parameters and is expressed as a numerical value ranging from 0 to 1. A higher score indicates a more stable and focused state, while a lower score suggests distraction or emotional variation. By introducing this

quantitative measure, the system provides a scientific approach to evaluating meditation quality, allowing users to track their performance objectively.

The project also aims to implement a real-time audio feedback mechanism that guides the user during meditation. When the system detects that the user is in a valid meditative state, characterized by closed eyes and high neutrality, it plays a soothing “Om” chanting sound. This audio feedback reinforces the user’s focus and encourages them to maintain stillness. If the system detects any disturbance, such as eye opening or changes in facial expression, the audio is stopped immediately. This immediate response creates an interactive feedback loop that helps users correct their behavior in real time.

Another objective of the project is to integrate multiple technologies into a single cohesive system. The system combines computer vision techniques for facial analysis, machine learning models for emotion detection, and embedded systems for deployment. This integration demonstrates the practical application of interdisciplinary concepts and highlights the ability to design complex systems that involve both hardware and software components working together.

The development of a user-friendly interface is also an important objective of the project. The system provides a simple and interactive interface that displays real-time information such as camera feed, eye state, neutrality score, and audio status. This allows users to easily understand their current state and interact with the system without difficulty. A well-designed interface enhances usability and ensures that the system can be used effectively by individuals with varying levels of technical knowledge.

In addition to real-time monitoring and feedback, the system aims to record and store session data for future analysis. This includes information such as session duration, neutrality scores over time, and eye state patterns. The data is stored in a lightweight format to ensure efficiency and can be used to track progress and identify trends in the user’s meditation practice. This data-driven approach enables users to evaluate their performance and make improvements over time.

Another significant objective is to implement the system on a Raspberry Pi platform, making it portable, cost-effective, and suitable for real-world deployment. The embedded implementation allows the system to function as a standalone device without requiring a full computer setup. This increases accessibility and makes the system practical for use in various environments, including homes, wellness centers, and therapy sessions.

The project also focuses on ensuring efficiency and optimization so that the system can operate smoothly on limited hardware resources. Techniques such as reducing frame resolution, using lightweight algorithms, and optimizing memory usage are applied to maintain real-time performance. This ensures that the system remains responsive and reliable even when running on a low-power device like Raspberry Pi.

Ultimately, the objective of this project is to enhance the overall meditation experience by providing real-time guidance, objective analysis, and performance tracking. By combining modern technologies with traditional meditation practices, the system helps users achieve deeper levels of focus, improve emotional stability, and develop consistency in their practice. Furthermore, the project lays the foundation for future advancements, including integration with mobile applications, cloud-based analytics, and more advanced artificial intelligence models, ensuring that the system can evolve and expand its capabilities over time.

The main objective is to create a reliable meditation assistant that tracks calmness through facial expressions and gives real-time feedback. It aims to support meditation practice by improving consistency, providing measurable performance, and helping users control their facial stillness and emotional stability.

CHAPTER 4

Key Features

The proposed meditation tracking system incorporates a wide range of advanced features that collectively enhance the overall meditation experience by providing real-time monitoring, analysis, and feedback. These features are designed to ensure accuracy, responsiveness, and user engagement while maintaining simplicity and efficiency.

One of the most important features of the system is **eye detection**, which plays a crucial role in determining whether the user is actively engaged in meditation. The system uses facial landmark detection techniques to identify key points around the eyes and calculates the Eye Aspect Ratio (EAR) to determine eye state. When the EAR value falls below a predefined threshold, the system classifies the eyes as closed, indicating a focused meditative state. Conversely, if the eyes are open, it suggests a possible loss of concentration. This feature ensures that the system can accurately monitor user attention in real time.

Another significant feature is **facial neutrality detection**, which evaluates whether the user's face remains calm and expressionless during meditation. A neutral facial expression is considered an indicator of emotional stability and inner calmness. The system analyzes multiple facial parameters such as mouth ratio, lip curvature, and eyebrow position to detect any deviations from neutrality. In addition to heuristic methods, the system also supports optional deep learning-based emotion detection using models such as DeepFace, which can classify emotions into categories like happy, sad, angry, or neutral. This combination of techniques improves the accuracy and reliability of emotional analysis.

The system also includes a powerful **audio feedback mechanism**, which is a key interactive feature. When the user maintains a valid meditative state—characterized by closed eyes and high facial neutrality—the system plays a soothing “Om” chanting sound. This audio acts as a reinforcement signal, helping the user maintain focus and deepen their meditation. If the system detects any disturbance, such as eye opening or changes in facial expression, the audio is stopped immediately. This real-time feedback loop ensures that users are constantly aware of their state and can quickly correct any distractions.

Another important feature is **real-time visualization**, which enhances user interaction and understanding. The system displays a live camera feed along with overlays of facial landmarks, allowing users to see how their facial features are being tracked. It also shows important metrics such as eye state, neutrality score, and emotion status. These visual indicators provide immediate insight into the user's condition and help in understanding how the system responds to different facial expressions.

The system further incorporates **expression stability analysis**, which monitors subtle changes in facial expressions over time. Instead of reacting to instant fluctuations, the system evaluates the consistency of facial features across multiple frames. This helps in reducing false detections caused by minor or temporary movements. By focusing on sustained changes rather than momentary variations, the system provides more accurate and stable feedback.

Another key feature is **session tracking and historical data storage**. The system records detailed

information about each meditation session, including duration, neutrality scores, and eye state patterns. This data is stored in a lightweight format such as JSON, making it easy to access and analyze. Users can review their past sessions to track progress, identify patterns, and improve their meditation practices over time. This feature adds a data-driven dimension to meditation, making it more structured and measurable.

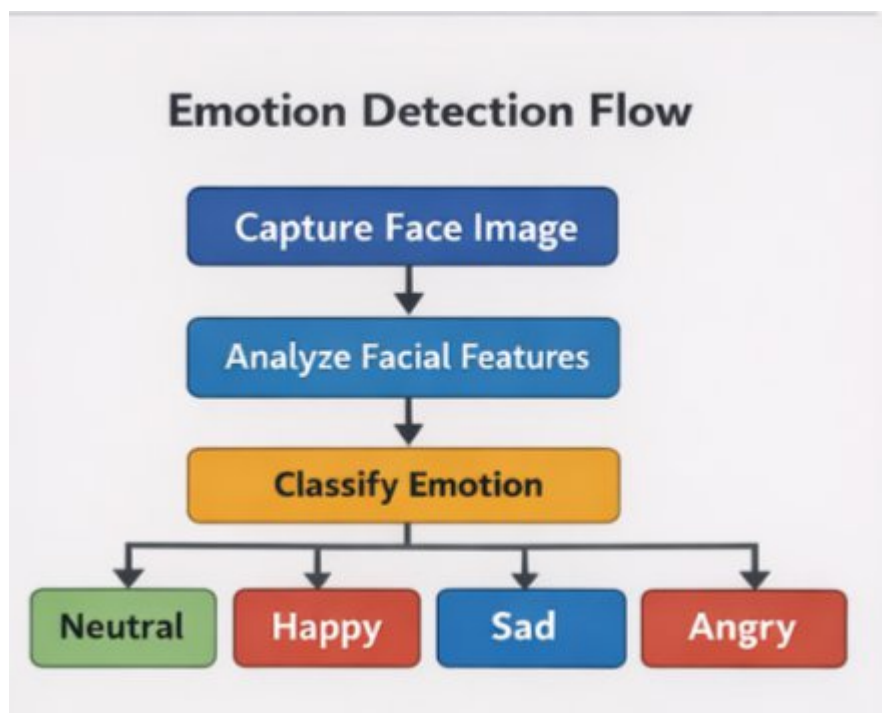
The system also offers **customization and configuration options**, allowing users to adjust various parameters according to their preferences. Users can modify thresholds for eye detection and neutrality, enable or disable deep learning-based emotion detection, and adjust audio settings such as volume. This flexibility ensures that the system can be tailored to different user needs and environments.

Another notable feature is the system's **real-time performance capability**, which ensures low latency and immediate response. The system processes each video frame instantly, enabling it to detect even small changes in facial expressions or eye movements. This responsiveness is essential for maintaining an effective feedback loop and ensuring that users receive timely guidance during meditation.

The system is also designed with **robust error handling and reliability** in mind. It can handle various real-world challenges such as temporary camera disconnections, low lighting conditions, and missing data. The system continues to operate smoothly even under suboptimal conditions, ensuring a consistent user experience.

Finally, one of the most significant features of the system is its **Raspberry Pi-based implementation**, which makes it portable and cost-effective. The system can operate as a standalone device without requiring a full computer setup. This enhances its usability and allows it to be deployed in a variety of environments, including homes, wellness centers, and therapy sessions.

Overall, the key features of this system work together to create a comprehensive and intelligent meditation assistant. By combining real-time monitoring, accurate analysis, interactive feedback, and data tracking, the system provides a structured and effective approach to improving meditation practice.



CHAPTER – 5

Detailed Module Description

The proposed meditation tracking system is designed using a modular architecture, where the overall functionality is divided into multiple interconnected components. Each module is responsible for performing a specific task, and together they work in coordination to achieve real-time monitoring, analysis, and feedback. This modular design not only improves the clarity and organization of the system but also enhances its scalability, maintainability, and flexibility.

In complex systems involving computer vision and real-time processing, dividing the system into smaller functional units is essential. It allows each component to be developed, tested, and optimized independently. In this project, the system is structured into key modules such as face detection, session tracking, audio management, data storage, and user interface. Each of these modules plays a crucial role in ensuring that the system operates efficiently and accurately.

The workflow of the system begins with capturing video input through a camera, which is then processed by the face detection module to identify and analyze facial features. The extracted data is passed to other modules for further processing, such as evaluating eye state, calculating neutrality score, and determining the user's meditation state. Based on this evaluation, the audio manager module generates appropriate feedback, while the session tracker and database modules record relevant data for analysis.

One of the key advantages of this modular approach is that it allows easy integration of new features and technologies. For example, advanced emotion detection models or improved tracking algorithms can be added to the system without significantly modifying existing components. Similarly, individual modules can be optimized or replaced independently, making the system more adaptable to future requirements.

Another important aspect of the modular design is fault isolation. If an error occurs in one module, it can be identified and resolved without affecting the entire system. This improves the robustness and reliability of the system, especially in real-world environments where conditions may vary.

The modules are designed to communicate with each other in a sequential pipeline, where the output of one module serves as the input for the next. This ensures a smooth flow of data and enables real-time processing. The system continuously cycles through this pipeline, capturing, analyzing, and responding to user behavior in an interactive manner.

Furthermore, the modular structure supports better performance optimization. Resource-intensive tasks can be optimized individually, and unnecessary computations can be minimized. This is particularly important for implementation on embedded platforms like Raspberry Pi, where computational resources are limited.

Overall, the modular design forms the backbone of the system, enabling efficient implementation, easy maintenance, and future scalability. The following subsections provide a detailed description of each module, explaining their functionality, working mechanisms, and role within the system.

5.1 Face Detector Module

The Face Detector Module forms the backbone of the entire system, as all subsequent computations and decisions depend on accurate detection and analysis of the user's face. This module is responsible for identifying the presence of a face in each video frame and extracting detailed facial features that are essential for determining the user's meditative state.

The module operates using the MediaPipe Face Mesh framework, which provides a highly efficient and accurate method for detecting facial landmarks. Unlike traditional face detection methods that only identify the bounding box of a face, MediaPipe extracts a dense set of approximately 468 facial landmarks. These landmarks represent critical points on the face, including the eyes, lips, eyebrows, nose, and jawline, enabling precise geometric analysis.

The working of this module begins with continuous video input from the camera. Each frame is processed individually, and the system checks for the presence of a face. If a face is detected, the landmark detection algorithm is applied to map the facial structure. If no face is detected, the frame is discarded to conserve computational resources. This selective processing improves system efficiency and ensures that unnecessary computations are avoided.

Once landmarks are extracted, the module performs feature extraction. It calculates parameters such as the Eye Aspect Ratio (EAR) to determine eye openness, and measures distances between key facial points to detect expressions. These features are crucial for identifying whether the user is maintaining a calm and focused state.

The module is designed to handle real-world challenges such as variations in lighting, slight head movements, and changes in camera angle. To improve robustness, the system may apply preprocessing techniques such as image normalization and smoothing. Additionally, temporal consistency is maintained by tracking facial landmarks across consecutive frames, reducing sudden fluctuations and improving stability.

One of the key challenges in this module is maintaining accuracy while ensuring real-time performance on limited hardware. To address this, optimized algorithms and efficient data structures are used. The module also avoids redundant calculations by reusing previously computed values when possible.

Overall, the Face Detector Module serves as the foundation of the system, providing accurate and reliable facial data that drives all subsequent analysis and decision-making processes.

5.2 Session Tracker Module

The Session Tracker Module is responsible for monitoring the progression of a meditation session and maintaining a comprehensive record of user activity. It plays a vital role in transforming the

system into a data-driven solution by capturing both real-time and historical information.

The module begins by initializing a session when the user starts meditation. It records the start time and continuously logs data throughout the session. Each frame processed by the system contributes to the session dataset, including parameters such as eye state, neutrality score, emotion classification, and audio feedback status.

A key feature of this module is its ability to perform temporal analysis. Instead of simply recording isolated values, the module tracks how these values change over time. This allows the system to identify patterns, such as periods of sustained focus or frequent distractions. By analyzing these trends, the system can provide meaningful insights into the user's meditation behavior.

The module also computes summary statistics at the end of each session. These include total session duration, average neutrality score, percentage of time with eyes closed, and number of interruptions. These metrics provide a clear and concise overview of the user's performance.

To ensure efficient storage, the module uses lightweight data formats such as JSON. Each session is stored as a separate file, containing both raw data and summary metrics. This structure allows for easy retrieval and analysis of past sessions.

Another important aspect of this module is its ability to handle multiple sessions. Users can build a history of their meditation practices, enabling long-term tracking and improvement. The module may also support data visualization, such as graphs showing neutrality over time, to provide a more intuitive understanding of performance.

Challenges in this module include managing large volumes of data and ensuring consistency across sessions. To address these issues, efficient data handling techniques and validation mechanisms are implemented.

Overall, the Session Tracker Module enhances the system by providing a structured and analytical approach to meditation, enabling users to monitor their progress and improve over time.

5.3 Audio Manager Module

The Audio Manager Module is responsible for providing real-time auditory feedback based on the user's meditation state. This module transforms the system from a passive monitoring tool into an interactive assistant that actively guides the user.

The primary function of this module is to control the playback of the "Om" chanting sound. The audio is triggered when the system detects that the user is in a valid meditative state, defined by closed eyes and high facial neutrality. The sound acts as a reinforcement signal, helping the user maintain focus and deepen their meditation.

The module continuously receives input from the decision-making component of the system. Based on this input, it determines whether to start, stop, or maintain audio playback. If a disturbance is detected, such as eye opening or expression change, the audio is stopped

immediately. This instant response ensures that the user is made aware of distractions in real time.

In addition to the main audio feedback, the module may generate auxiliary sounds, such as alerts or beeps, to indicate specific events. For example, a short beep may be played when the user opens their eyes, providing immediate feedback.

The implementation of this module uses audio libraries such as Pygame or Pydub, which provide efficient and flexible audio handling capabilities. The module is optimized to minimize latency, ensuring that audio responses are synchronized with the user's actions.

One of the challenges in this module is ensuring smooth audio playback without interruptions or delays, especially on resource-constrained devices. To address this, the system uses efficient buffering techniques and prioritizes audio processes.

The module also allows for customization, enabling users to adjust volume levels or select different audio tones. This enhances user experience and adaptability.

Overall, the Audio Manager Module plays a critical role in guiding the user through real-time feedback, making the meditation process more interactive and effective.

5.4 Face Database Module

The Face Database Module is responsible for storing, managing, and retrieving all data generated by the system. It serves as the backbone for data persistence and analysis, enabling the system to maintain a record of user activity over time.

This module stores detailed information about each session, including timestamps, neutrality scores, eye state data, and audio activity. The data is organized in a structured format, allowing for easy access and analysis.

The use of JSON as a storage format provides several advantages. It is lightweight, easy to read, and compatible with various programming environments. This makes it ideal for embedded systems like Raspberry Pi, where resources are limited.

The module ensures data integrity by implementing proper file handling mechanisms. It prevents data corruption and ensures that all session data is stored accurately. It also supports data retrieval functions, allowing users to access past sessions and analyze their performance.

Another important feature of this module is its ability to support data analysis and visualization. The stored data can be used to generate graphs and reports, providing insights into trends and patterns. For example, users can view how their neutrality score changes over time or identify periods of frequent distraction.

Challenges in this module include managing storage space and ensuring efficient data retrieval. To address these issues, the system may implement data compression or periodic cleanup of old data.

Overall, the Face Database Module adds significant value to the system by enabling long-term tracking and analysis of meditation performance.

5.5 Streamlit Interface (Main Application)

The Streamlit Interface serves as the central hub of the system, integrating all modules and providing a user-friendly interface for interaction. It acts as the bridge between the system's internal processing and the user, ensuring that information is presented clearly and intuitively.

The interface displays the live camera feed along with overlays of facial landmarks, allowing users to see how their face is being analyzed in real time. It also shows key metrics such as eye state, neutrality score, emotion status, and audio feedback status.

The interface includes interactive controls that allow users to start or stop sessions, adjust thresholds, and customize system settings. This flexibility ensures that the system can be adapted to different user preferences and environments.

Streamlit is chosen for its simplicity and ability to create web-based applications quickly. The interface runs locally and can be accessed through a web browser, making it easy to use without additional setup.

The interface also integrates with the Session Tracker and Database modules to display session summaries and historical data. This provides users with a complete overview of their performance.

One of the challenges in designing the interface is ensuring responsiveness and clarity while displaying multiple data elements. To address this, the interface is designed with a clean layout and efficient data updates.

Overall, the Streamlit Interface enhances user experience by providing real-time visualization, control, and feedback, making the system accessible and effective for users of all levels.

CHAPTER – 6

Feature Breakdown

The proposed meditation tracking system incorporates multiple advanced features that work together to provide an accurate, responsive, and interactive meditation experience. These features are carefully designed to monitor the user's physical and emotional state in real time and translate this information into meaningful feedback. Each feature contributes to a specific aspect of the system, such as detecting user focus, analyzing emotional stability, or providing guidance through audio and visual cues.

In modern intelligent systems, especially those involving real-time computer vision, it is essential to break down functionality into individual features that can be analyzed and optimized independently. This approach not only improves system performance but also enhances clarity in understanding how different components contribute to the overall functionality. In this project, the feature breakdown provides a detailed explanation of the key mechanisms that enable the system to track meditation effectively.

The system primarily focuses on monitoring facial expressions and eye behavior, as these are reliable indicators of a person's mental state during meditation. By analyzing these physical cues, the system can determine whether the user is maintaining focus or experiencing distractions. These observations are then converted into measurable parameters, such as eye state and neutrality score, which form the basis for decision-making.

Another important aspect of the system is its ability to operate in real time. Each feature is designed to process data continuously and respond instantly to any changes in the user's condition. This ensures that the system can provide immediate feedback, helping users stay aware of their actions and maintain a consistent meditative state. The combination of real-time processing and intelligent analysis makes the system highly effective and responsive.

The features of the system are not isolated; instead, they are interconnected and work as part of a unified pipeline. For example, eye detection and facial neutrality analysis together determine whether the user is in a valid meditative state. This decision then triggers the audio feedback system, which provides guidance to the user. Similarly, real-time visualization and session storage features enhance user interaction and allow long-term performance tracking.

In addition to functionality, the features are designed with efficiency and reliability in mind. The system uses optimized algorithms and lightweight processing techniques to ensure smooth performance, even on resource-constrained devices like Raspberry Pi. Error handling mechanisms are also incorporated to maintain stability under varying conditions, such as changes in lighting or temporary loss of camera input.

Overall, the feature breakdown provides a comprehensive understanding of how individual functionalities contribute to the system's operation. The following subsections describe each feature in detail, explaining their working principles, significance, and role in enhancing the meditation experience.

6.1 Eye Detection

Eye detection is one of the most critical features of the proposed system, as it directly indicates whether the user is actively engaged in meditation. During meditation, keeping the eyes closed is a fundamental requirement for maintaining focus and minimizing external distractions. Therefore, accurately detecting eye closure is essential for determining the user's meditative state.

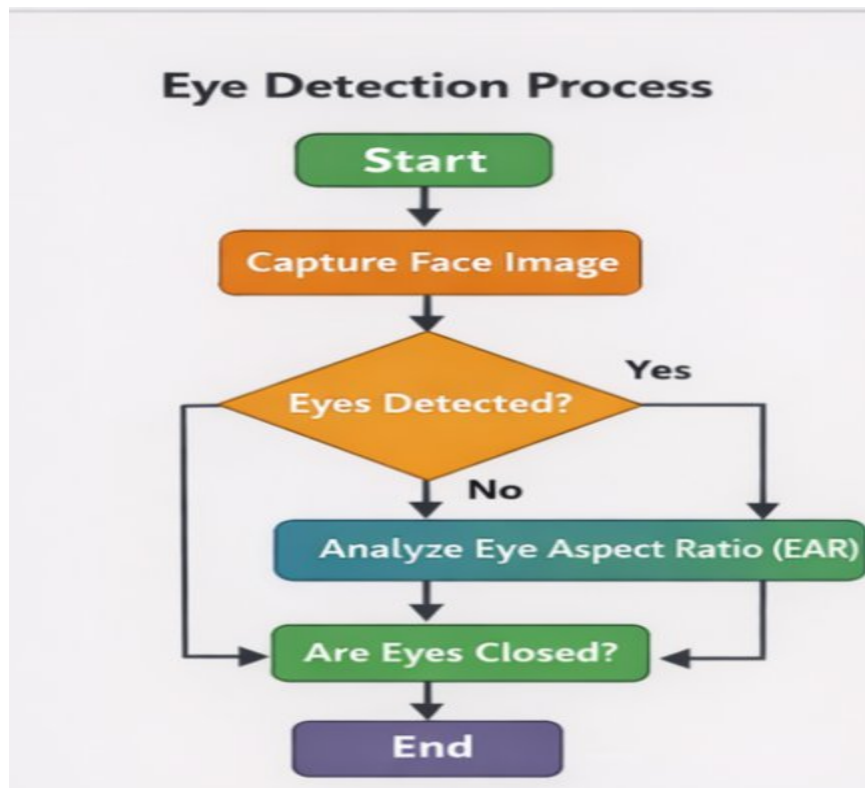
The system uses facial landmark detection techniques to identify key points around the eyes. These landmarks are extracted using MediaPipe Face Mesh, which provides a detailed representation of facial geometry. Based on these landmarks, the system calculates the Eye Aspect Ratio (EAR), a mathematical measure that represents the openness of the eye.

The EAR is computed using distances between vertical and horizontal eye landmarks. When the eyes are open, the vertical distance between eyelids is relatively high, resulting in a higher EAR value. When the eyes are closed, this vertical distance decreases significantly, causing the EAR value to drop below a predefined threshold.

To ensure accuracy, the system continuously monitors the EAR across multiple frames. Instead of relying on a single frame, it uses temporal smoothing techniques to avoid false detections caused by blinking or noise. This ensures that brief eye closures or random fluctuations do not trigger incorrect responses.

The system also allows threshold customization, enabling users to adjust sensitivity based on individual differences such as eye shape and camera angle. This flexibility improves usability and accuracy across different users.

Overall, the eye detection feature provides a reliable and real-time method for monitoring user attention, forming a key component of the meditation tracking system.



6.2 Neutrality & Emotion Detection

Facial neutrality detection is another essential feature of the system, as it reflects the user's emotional stability during meditation. A calm and neutral facial expression indicates a relaxed mental state, while changes in expression may suggest distraction or emotional activity.

The system evaluates facial neutrality by analyzing multiple facial features, including mouth shape, lip curvature, and eyebrow position. These features are extracted from facial landmarks and used to detect subtle changes in expression. For example, a smile or frown can be identified by measuring the curvature of the lips and the ratio of mouth width to height.

In addition to heuristic methods, the system supports deep learning-based emotion detection using models such as DeepFace. This allows the system to classify facial expressions into categories such as neutral, happy, sad, angry, and surprised. By combining heuristic and deep learning approaches, the system achieves higher accuracy and robustness.

The neutrality score is calculated by combining multiple facial parameters using weighted averaging. The score is normalized to a range between 0 and 1, where higher values indicate a more neutral and stable expression. This numerical representation allows for objective evaluation of the user's emotional state.

To improve reliability, the system applies smoothing techniques to reduce noise and avoid sudden fluctuations. This ensures that only sustained changes in expression are considered significant.

This feature plays a crucial role in ensuring that the user maintains emotional stability during meditation, contributing to a more effective and focused experience.



6.3 Audio Feedback System

The audio feedback system is one of the most interactive features of the project, providing real-time guidance to the user during meditation. It acts as a bridge between system analysis and user behavior, ensuring continuous engagement.

The system plays a soothing “Om” chanting sound when the user meets the required conditions for meditation, such as closed eyes and high neutrality score. This audio serves as a reinforcement signal, encouraging the user to maintain focus and stillness.

If the system detects any disturbance, such as eye opening or changes in facial expression, the audio is stopped immediately. This instant feedback helps users become aware of distractions and correct their behavior in real time.

The system may also include additional audio cues, such as beeps or alerts, to indicate specific events like session start, end, or state changes. These cues enhance user interaction and provide clear feedback.

The audio system is implemented using efficient libraries such as Pygame, ensuring low latency and smooth playback. Optimization techniques are used to minimize delays and ensure synchronization with system decisions.

Overall, the audio feedback system transforms the project into an interactive meditation assistant, providing continuous guidance and improving user experience.



6.4 Expression Stability Analysis

Expression stability analysis focuses on evaluating the consistency of facial expressions over time. Instead of reacting to instant changes, the system analyzes trends across multiple frames to determine whether the user's expression is stable.

This feature is important because minor facial movements, such as slight muscle twitches or involuntary expressions, can occur naturally and should not be treated as distractions. By analyzing temporal consistency, the system distinguishes between meaningful changes and random fluctuations.

The system uses a smoothing buffer that stores recent values of facial parameters and computes an average over time. This reduces noise and improves reliability. Sudden spikes or drops in values are filtered out, ensuring that only sustained changes affect the system's decision.

Expression stability analysis enhances the accuracy of neutrality detection and reduces false positives. It ensures that the system responds only to significant changes, providing a more stable and realistic evaluation of the user's state.

6.5 Real-Time Visualization

Real-time visualization is a key feature that enhances user interaction and understanding of the system. The interface displays a live video feed along with overlays of facial landmarks, allowing users to see how their face is being tracked.

In addition to the video feed, the system displays important metrics such as eye state, neutrality score, emotion classification, and audio status. These metrics are updated continuously, providing immediate feedback to the user.

The visualization helps users understand how their actions affect the system's response. For example, opening the eyes or changing expressions will immediately reflect in the displayed metrics and audio feedback.

The interface is designed using Streamlit, which provides a simple and interactive platform for displaying real-time data. The layout is clean and user-friendly, ensuring that users can easily interpret the information.

6.6 Session Historical Storage

Session historical storage is an important feature that enables long-term tracking and analysis of meditation performance. The system records detailed data for each session, including timestamps, neutrality scores, eye state patterns, and audio activity.

The data is stored in a lightweight format such as JSON, ensuring efficient storage and easy retrieval. Each session is saved as a separate file, allowing users to review individual sessions or compare multiple sessions.

over time.

This feature allows users to analyze trends, identify patterns, and track improvements in their meditation practice. For example, users can observe whether their neutrality score improves over time or whether they are able to maintain eye closure for longer durations.

The stored data can also be used for generating visualizations such as graphs and charts, providing a more intuitive understanding of performance.

Session historical storage transforms meditation into a measurable and data-driven activity, enabling users to achieve continuous improvement.

CHAPTER – 7

Algorithms Used

The performance and reliability of the proposed meditation tracking system are fundamentally dependent on the algorithms employed for analyzing facial features and interpreting user behavior. Since the system operates in real time and relies on continuous video input, it is essential to utilize algorithms that are not only accurate but also computationally efficient. These algorithms form the core intelligence of the system, enabling it to transform raw visual data into meaningful insights about the user's meditative state.

In real-world computer vision applications, images captured from a camera contain a large amount of unstructured data that does not directly convey useful information. Therefore, specialized algorithms are required to extract relevant features such as eye openness, facial expressions, and emotional cues. In this system, these features are derived using geometric relationships between facial landmarks, statistical analysis, and machine learning techniques. The extracted features are then converted into measurable parameters that can be evaluated systematically.

A key aspect of the algorithmic design in this project is the use of lightweight yet effective methods. Since the system is deployed on a Raspberry Pi, which has limited computational resources, the algorithms must be optimized for speed and efficiency. For this reason, geometric algorithms such as Eye Aspect Ratio (EAR) and mouth ratio calculations are used for primary detection tasks. These methods are fast, reliable, and suitable for real-time processing. At the same time, the system also supports more advanced techniques, such as deep learning-based emotion detection, which can be enabled when higher accuracy is required.

Another important consideration is robustness. The algorithms must be capable of handling variations in lighting conditions, camera angles, facial orientations, and user-specific differences. To achieve this, the system incorporates normalization techniques and threshold-based decision-making. Additionally, temporal smoothing is applied to reduce noise and prevent sudden fluctuations from affecting the results. Instead of relying on a single frame, the system analyzes multiple consecutive frames to ensure stable and reliable outputs.

The algorithms used in this system are also designed to work in a sequential pipeline. Each stage of processing builds upon the results of the previous stage. For example, facial landmark detection provides the necessary data for computing eye and mouth features, which are then used to calculate the neutrality score. This score, along with eye state, is evaluated using decision logic to determine whether the user is in a valid meditative state. Based on this evaluation, the system generates appropriate feedback.

Furthermore, the integration of multiple algorithms enhances the overall accuracy of the system. By combining geometric methods with machine learning techniques, the system achieves a balance between efficiency and precision. This hybrid approach ensures that the system remains responsive while still providing reliable results.

Overall, the algorithms used in this project play a critical role in enabling intelligent and real-time meditation tracking. They provide the foundation for feature extraction, analysis, and decision-making,

ensuring that the system operates effectively under various conditions. The following subsections describe each algorithm in detail, including their working principles, mathematical formulations, and significance within the system.

7.1 Eye Aspect Ratio (EYE) Algorithm

One of the fundamental algorithms used in the system is the Eye Aspect Ratio (EAR), which is employed to detect whether the user's eyes are open or closed. Eye closure is a critical indicator of meditation, as it reflects focus and internal awareness.

The EAR is calculated using distances between specific facial landmarks around the eye. These landmarks are obtained through facial landmark detection techniques such as MediaPipe. The EAR is defined as the ratio of vertical distances to horizontal distance of the eye.

$$EAR = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2 \cdot \|p_1 - p_4\|}$$

When the eyes are open, the vertical distances are relatively large, resulting in a higher EAR value. When the eyes are closed, these distances decrease significantly, causing the EAR to drop below a predefined threshold.

To improve reliability, the system does not rely on a single frame. Instead, it monitors EAR values over multiple frames and applies smoothing techniques to avoid false detections caused by blinking or noise. A threshold value is defined, and if the EAR remains below this threshold for a certain duration, the system classifies the eyes as closed.

This algorithm is lightweight and computationally efficient, making it suitable for real-time applications on embedded systems like Raspberry Pi.

7.2 Mouth Ratio & Smile Detection

The system also uses a mouth ratio algorithm to detect changes in facial expressions, particularly smiling or tension. This helps in identifying deviations from a neutral facial state.

The mouth ratio is calculated by comparing the width of the mouth to its height. A higher ratio typically indicates a smile or stretched expression, while a lower ratio suggests a relaxed or neutral state.

The algorithm works by extracting landmarks around the lips and computing distances between key points. These distances are then used to calculate the ratio, which serves as an indicator of expression.

By continuously monitoring the mouth ratio, the system can detect subtle changes in facial expressions that may indicate emotional activity or distraction. This information is used in combination with other features to determine overall facial neutrality.

7.3 Facial Emotion Detection

To enhance the accuracy of expression analysis, the system incorporates a facial emotion detection algorithm. This algorithm uses deep learning models, such as those provided by DeepFace, to classify facial expressions into predefined emotion categories.

The model processes facial images and extracts high-level features using convolutional neural networks (CNNs). These features are then used to classify emotions such as neutral, happy, sad, angry, or surprised.

Emotion detection provides additional context for evaluating the user's state. For example, a neutral expression indicates calmness, while expressions such as happiness or anger may indicate distraction. This helps the system make more informed decisions.

Although deep learning models provide high accuracy, they are computationally intensive. Therefore, this feature is optional and can be disabled on low-power devices to maintain real-time performance.

7.4 Om Sound Triggering Logic

The system uses a rule-based decision algorithm to determine when to play or stop the “Om” chanting sound. This logic is based on the outputs of the EAR algorithm and neutrality score calculation.

The conditions for triggering audio are:

- Eyes must be closed (EAR below threshold)
- Neutrality score must be above a predefined threshold

If both conditions are satisfied, the system plays the audio. If either condition is violated, the audio is stopped immediately.

This decision-making process is implemented using simple conditional statements, ensuring fast execution and low computational overhead. The logic is continuously evaluated in real time, allowing the system to respond instantly to changes in the user's state.

```
1 import tensorflow as tf
2 from tensorflow import keras
3 from tensorflow.keras import layers, models
4 import matplotlib.pyplot as plt
5
6 # -----
7 # Paths and Setup
8 # -----
9 train_dir = "fer2013/train"
10 test_dir = "fer2013/test"
11
12 IMG_SIZE = (48, 48)
13 BATCH_SIZE = 64
14
15 # -----
16 # Load Data
17 # -----
18 train_ds = keras.preprocessing.image_dataset_from_directory(
19     train_dir,
20     image_size=IMG_SIZE,
21     color_mode="rgb",
22     batch_size=BATCH_SIZE
23 )
24 val_ds = keras.preprocessing.image_dataset_from_directory(
25     test_dir,
26     image_size=IMG_SIZE,
27     color_mode="rgb",
28     batch_size=BATCH_SIZE
29 )
30
31 # Get class names (outside tf graph)
32 class_names = train_ds.class_names
33 print("Class names:", class_names)
34
35 # Identify neutral label index
36 if "neutral" in class_names:
```

CHAPTER – 8

User Interface Design

The user interface is an essential component of the proposed meditation tracking system, as it serves as the primary point of interaction between the user and the underlying system. While the backend of the system performs complex tasks such as facial detection, feature extraction, and decision-making, the interface ensures that these processes are presented in a clear, understandable, and user-friendly manner. It acts as a bridge that translates technical computations into meaningful visual and interactive elements.

In systems that involve real-time monitoring and feedback, the role of the interface becomes even more critical. The user must be able to observe system responses instantly and understand how their actions influence the output. In the context of meditation, this is particularly important because the system is designed to guide users toward maintaining focus and emotional stability. Therefore, the interface must provide continuous feedback without causing distraction or interrupting the meditation process.

The design of the interface in this project is guided by the need to balance functionality and simplicity. While it is important to display relevant information such as eye state, neutrality score, and audio status, it is equally important to ensure that the interface remains minimal and non-intrusive. A cluttered or overly complex interface could negatively impact the user's ability to concentrate. Hence, the interface is carefully structured to present only essential information in a clean and organized layout.

Another important aspect of the interface is its real-time nature. The system processes video frames continuously, and the interface reflects these changes instantly. This requires efficient synchronization between the backend processing modules and the frontend display. The interface must be capable of updating multiple elements simultaneously, including live video feed, numerical metrics, and status indicators, without introducing delays or inconsistencies.

The interface is also designed to support user interaction and customization. Users can adjust system parameters such as detection thresholds and audio settings according to their preferences. This flexibility ensures that the system can adapt to different users and environments, improving overall usability and effectiveness.

Furthermore, the interface plays a significant role in enhancing user engagement and trust. By providing visual representations of facial landmarks and real-time metrics, it allows users to understand how the system interprets their facial expressions. This transparency helps build confidence in the system's accuracy and reliability.

Overall, the user interface is not merely a display component but an integral part of the system that enhances usability, provides feedback, and supports the overall goal of improving meditation practices. The following subsections describe the design principles, layout, and functionalities of the interface in detail.

8.1 Introduction to User Interface Design

The user interface (UI) of the proposed meditation tracking system plays a crucial role in bridging the gap between complex backend processing and user interaction. While the system internally performs advanced operations such as facial landmark detection, emotion analysis, and decision-making, the interface ensures that this information is presented in a simple, understandable, and user-friendly manner. A well-designed interface is essential for enabling users to interact with the system effectively without requiring technical knowledge.

In the context of meditation, the design of the interface must consider the psychological state of the user. Meditation is a process that requires calmness, focus, and minimal external distractions. Therefore, the interface is designed to be clean, minimalistic, and non-intrusive. It avoids excessive graphical elements, animations, or clutter that could distract the user. Instead, it focuses on presenting only the most relevant information in a clear and organized way.

The interface is developed using Streamlit, a Python-based framework that allows rapid creation of interactive web applications. Streamlit is particularly suitable for this project because it supports real-time data updates and easy integration with backend processing. It enables the display of live video feeds, dynamic metrics, and user controls within a single unified interface.

Another important aspect of the UI is its role in providing feedback. The system continuously analyzes the user's facial expressions and eye behavior, and the interface reflects these changes in real time. This helps users understand how their actions affect the system's response. For example, if the user opens their eyes, the interface immediately updates the eye state and stops the audio feedback, providing instant awareness.

Overall, the user interface is designed not just as a display system, but as an interactive component that enhances the meditation experience. It ensures that users can easily understand their current state, receive guidance, and improve their practice over time.

8.2 Design Principles and Objectives

The design of the user interface is guided by several important principles that ensure usability, efficiency, and user satisfaction. These principles are particularly important in a meditation-based application, where the interface must support a calm and focused environment.

One of the primary design principles is simplicity. The interface avoids unnecessary complexity and presents only essential information. This reduces cognitive load and allows users to focus on meditation rather than interacting with the system. A simple interface also ensures that users of all experience levels can use the system without difficulty.

Clarity is another key principle. All information displayed on the interface is clearly labeled and easy to interpret. Metrics such as eye state and neutrality score are presented in a straightforward manner, often accompanied by color indicators to enhance understanding. This ensures that users

can quickly grasp their current state without confusion.

Responsiveness is also critical. The interface must update in real time to reflect changes in the user's state. Any delay in updating information could reduce the effectiveness of the feedback system. Therefore, the interface is designed to process and display data instantly.

User-friendliness is achieved by designing intuitive controls and layouts. Buttons, sliders, and other interactive elements are placed logically and labeled clearly. This allows users to easily navigate and control the system.

Finally, the interface is designed to minimize distraction. Since meditation requires focus, the interface uses soft colors, simple layouts, and minimal animations. This ensures that the interface supports the meditation process rather than interfering with it.

8.3 Overall Layout of the Interface

The overall layout of the interface is carefully structured to organize information in a clear and logical manner. A well-organized layout ensures that users can easily access different functionalities without confusion or effort.

The interface is divided into multiple sections, each serving a specific purpose. The sidebar is used for controls and configuration settings, while the main display panel shows the live video feed. Additional sections are used to display real-time metrics and session summaries.

The central area of the interface is dedicated to the live video feed, as it is the most important component. This ensures that users can clearly see how their face is being analyzed. Surrounding this central area are panels that display relevant information such as eye state and neutrality score.

The sidebar is positioned on one side of the interface, providing easy access to controls without interfering with the main display. This separation of controls and display ensures a clean and organized layout.

Spacing and alignment are also important aspects of the layout. Proper spacing between elements improves readability and prevents the interface from appearing cluttered. Consistent alignment ensures a professional and visually appealing design.

The layout is designed to be responsive, meaning it can adapt to different screen sizes and resolutions. This ensures that the interface remains usable on various devices.

Overall, the layout design focuses on clarity, organization, and ease of use, ensuring that users can interact with the system efficiently.

8.4 Sidebar Controls

The sidebar serves as the primary control panel of the application, allowing users to interact with the system and customize its behavior. It is designed to provide all necessary controls in a compact and accessible format.

The sidebar includes controls for starting and stopping meditation sessions. These controls are essential for managing the operation of the system. Users can initiate a session with a single click and stop it when they are finished.

Another important feature of the sidebar is threshold adjustment. Users can modify parameters such as the Eye Aspect Ratio threshold and neutrality score threshold. This allows the system to be customized according to individual preferences and environmental conditions.

The sidebar also includes options for enabling or disabling emotion detection. Since deep learning-based emotion detection can be computationally intensive, users can choose whether to use this feature based on their system capabilities.

Audio controls are also provided, allowing users to adjust volume or enable/disable sound. This ensures that the audio feedback system can be tailored to the user's comfort level.

The design of the sidebar focuses on simplicity and usability. Controls are grouped logically, and sliders are used for easy adjustment. Labels are clear and descriptive, ensuring that users can understand each control without difficulty.

8.5 Main Display Panel

The main display panel is the central component of the interface, where the live camera feed is shown. It provides real-time visualization of the user's face and the system's analysis.

The panel displays the video feed captured from the camera, along with overlays of facial landmarks. These landmarks highlight key facial features such as eyes, lips, and eyebrows, showing how the system tracks the face.

This visualization is important for transparency, as it allows users to see how the system is analyzing their face. It also helps in debugging and understanding system behavior.

The panel may also highlight specific regions, such as the eyes, to indicate where calculations are being performed. This provides additional insight into the system's operation.

The design of the panel ensures that the video feed is clear and unobstructed. Overlays are subtle and do not interfere with visibility.

Overall, the main display panel plays a crucial role in providing real-time feedback and enhancing user understanding.

8.6 Real-Time Metrics Display

The real-time metrics display is one of the most important components of the user interface, as it provides

continuous feedback about the user's current meditation state. These metrics translate complex backend computations into simple, understandable values that users can easily interpret during their session.

The interface displays multiple parameters such as eye state, neutrality score, emotion status, and audio status. Each of these metrics represents a specific aspect of the user's behavior. For example, the eye state indicates whether the user's eyes are open or closed, which is a direct measure of focus. Similarly, the neutrality score reflects the level of emotional stability and calmness.

To improve readability, the metrics are presented using clear labels and visual cues. Color coding is used extensively, where green indicates a stable or desired state, and red indicates a disturbance or issue. This allows users to quickly understand their condition without needing to analyze numerical values in detail.

The neutrality score is typically displayed as a decimal value between 0 and 1, providing a quantitative measure of the user's state. This helps users track their progress and understand how small changes in their facial expressions affect the system's evaluation.

The system ensures that these metrics are updated in real time, with minimal delay. This is achieved through efficient data processing and synchronization with the backend. Continuous updates allow users to receive immediate feedback, which is essential for maintaining focus during meditation.

Overall, the real-time metrics display enhances user awareness and provides valuable insights into their meditation performance, making it a critical component of the interface.

8.7 Audio Feedback Indicator

The audio feedback indicator is a visual representation of the system's audio output, providing users with additional clarity about the system's state. While the primary feedback is delivered through sound, the visual indicator ensures that users can understand the system's behavior even in situations where audio may not be clearly audible.

This indicator shows whether the "Om" chanting sound is currently active or inactive. When the user is in a valid meditative state, the indicator changes to a positive state, often represented by a green color or a "Playing" label. Conversely, when the system detects a disturbance, such as eye opening or expression change, the indicator switches to a negative state, represented by a red color or a "Stopped" label.

The inclusion of this feature enhances transparency, as users can easily correlate their actions with the system's response. For example, if the user opens their eyes and the audio stops, the visual indicator immediately reflects this change, reinforcing the feedback loop.

The design of the indicator is kept simple and unobtrusive, ensuring that it does not distract the user. It is usually placed near the real-time metrics for easy visibility.

In addition to basic status display, the indicator can also be extended to show transitions, such as "Starting" or "Stopping," providing more detailed feedback about system behavior.

Overall, the audio feedback indicator complements the audio system by providing a clear and immediate

visual cue, enhancing user understanding and interaction.

8.8 Session Information Display

The session information display provides a summary of the user's meditation session, offering valuable insights into their performance. This feature transforms the system from a real-time monitoring tool into a comprehensive analysis platform.

During and after each session, the interface displays key statistics such as session duration, average neutrality score, number of eye openings, and total time spent in a valid meditative state. These metrics help users evaluate their performance objectively.

The session duration indicates how long the user was engaged in meditation, while the neutrality score provides a measure of emotional stability throughout the session. The number of eye openings reflects how often the user lost focus, and the total meditation time indicates the duration of sustained focus.

The information is presented in a clear and organized manner, often using tables or simple text summaries. In some cases, graphical representations such as charts or progress bars may be used to enhance understanding.

This feature encourages users to track their progress over time and identify areas for improvement. By comparing multiple sessions, users can observe trends and measure their development.

Overall, the session information display adds a data-driven dimension to meditation, making it more structured and goal-oriented.

8.9 Real-Time Data Updating Mechanism

The real-time data updating mechanism is responsible for ensuring that the interface reflects the latest system outputs without delay. This is a critical requirement for maintaining the effectiveness of the feedback system.

The interface is built using Streamlit, which automatically reruns scripts whenever new data is available. This allows the system to update metrics, video feeds, and indicators dynamically.

One of the key challenges in real-time updating is maintaining smooth transitions without flickering or lag. To address this, the system uses efficient data handling techniques and optimized rendering processes. Only necessary components are updated, reducing computational overhead.

Synchronization between the backend and frontend is also important. The system ensures that data generated by the processing modules is immediately reflected in the interface, maintaining consistency. Buffering techniques may be used to manage data flow and prevent sudden spikes or inconsistencies. This ensures stable and reliable updates.

Overall, the real-time updating mechanism ensures that users receive instant feedback, which is essential for maintaining focus and improving meditation performance.

8.10 User Interaction Flow

The user interaction flow defines how users interact with the system from start to finish. A well-designed interaction flow ensures ease of use and minimizes confusion.

The process begins when the user launches the application. The interface loads and displays the camera feed along with default settings. The user can then adjust parameters such as thresholds and audio settings using the sidebar.

Once the user starts the session, the system begins real-time monitoring and feedback. The user observes the interface and adjusts their behavior accordingly. The system continuously provides feedback through both visual and audio means.

At the end of the session, the user stops the system and views the session summary. This allows them to evaluate their performance and identify areas for improvement.

The interaction flow is designed to be simple and intuitive, requiring minimal effort from the user. This ensures that the system can be used effectively by individuals with different levels of technical expertise.



8.11 Error Handling in Interface

Error handling is an essential aspect of the user interface, ensuring that the system remains reliable and user-friendly even under unexpected conditions.

The interface is designed to detect and handle common issues such as camera disconnection, low lighting conditions, and failure to detect a face. When such issues occur, the system displays clear and informative messages to guide the user.

For example, if the camera is not detected, the interface may display a message prompting the user to check the connection. If lighting conditions are poor, the system may suggest improving illumination.

These messages are designed to be simple and easy to understand, avoiding technical jargon. This ensures that users can resolve issues without difficulty.

The system also ensures that errors do not cause crashes or interruptions. Instead, it continues to operate smoothly and attempts to recover from errors.

Overall, effective error handling improves system reliability and enhances user experience.

8.12 Performance Optimization in UI

Performance optimization is crucial for ensuring that the interface runs smoothly, especially on resource-constrained devices like Raspberry Pi.

The interface is designed to minimize computational overhead by using lightweight components and efficient rendering techniques. Only essential elements are displayed, and unnecessary graphics are avoided.

Video rendering is optimized to ensure smooth playback without lag. Data updates are handled efficiently to prevent delays.

The system also reduces the frequency of updates when necessary to maintain performance. This ensures that the interface remains responsive even under heavy load.

Overall, performance optimization ensures a smooth and seamless user experience.

8.13 Accessibility and Usability Considerations

The accessibility and usability of the user interface are critical factors that determine how effectively users can interact with the meditation tracking system. Since the system is intended for a wide range of users, including beginners and individuals with limited technical knowledge, the interface is designed to be intuitive, inclusive, and easy to navigate. The primary goal is to ensure that users can operate the system effortlessly without requiring extensive instructions or prior experience.

One of the key aspects of usability is simplicity. The interface avoids complex layouts and unnecessary elements, presenting only essential information in a clear and structured manner. This reduces cognitive load and allows users to focus on meditation rather than trying to understand the interface. The use of clear labels, simple language, and consistent terminology ensures that users can easily interpret the displayed information.

Readability is another important consideration. The interface uses appropriate font sizes, spacing, and contrast to ensure that all text is easily readable under different conditions. Color schemes are carefully selected to maintain visual comfort, especially since meditation sessions may last for extended periods. High-contrast text and background combinations are used to improve visibility, while avoiding overly bright or distracting colors.

The interface also incorporates intuitive controls that require minimal effort to operate. Buttons, sliders, and toggles are designed to be self-explanatory, with clear labels indicating their functionality. Default values are provided for most settings, allowing users to start using the system without making complex adjustments. This is particularly beneficial for beginners who may not be familiar with technical

parameters such as thresholds.

Another important aspect of accessibility is adaptability. The interface is designed to function across different devices and screen sizes, ensuring that it remains usable on various platforms. The layout adjusts dynamically to maintain readability and usability, regardless of the display environment.

The system also considers users with different levels of physical and cognitive abilities. For example, visual feedback is complemented with audio feedback, ensuring that users can understand system responses through multiple channels. This multimodal approach enhances accessibility and ensures that the system can be used effectively by a diverse group of users.

Error tolerance is another key usability feature. The interface is designed to handle incorrect inputs or unexpected situations gracefully, providing helpful messages and guidance instead of causing confusion. This ensures that users can recover from errors easily without frustration.

Overall, the accessibility and usability considerations ensure that the system is not only technically functional but also user-friendly and inclusive. By focusing on simplicity, clarity, and adaptability, the interface provides a comfortable and effective user experience.

8.14 Limitations and Future Improvements

While the user interface of the proposed system is designed to be efficient and user-friendly, it is important to acknowledge certain limitations that may affect its performance and usability. Identifying these limitations helps in understanding the scope of improvement and provides direction for future enhancements.

One of the primary limitations is related to the use of Streamlit as the development framework. Although Streamlit offers simplicity and rapid development, it has certain constraints in terms of advanced graphical customization and user interface flexibility. Complex animations, highly interactive dashboards, and advanced UI components are difficult to implement compared to dedicated frontend frameworks such as React or Angular. This may limit the visual appeal and interactivity of the interface.

Another limitation is the dependency on web browsers. Since the interface runs as a web-based application, its performance may vary depending on the browser and system configuration. In some cases, slower browsers or limited system resources may result in reduced responsiveness or delays in real-time updates.

The interface is also dependent on hardware performance, particularly when running on Raspberry Pi. Limited processing power can affect the smooth rendering of video and real-time updates, especially when multiple features such as emotion detection are enabled simultaneously. This may lead to occasional lag or reduced frame rates.

In addition, the current interface provides basic visualization features. While it displays essential metrics and live video feed effectively, it lacks advanced data visualization tools such as detailed graphs, trend analysis dashboards, and interactive reports. These features could provide deeper insights into user performance.

Another limitation is the lack of mobile optimization. Although the interface can be accessed through a browser, it is not specifically designed for mobile devices. This may affect usability on smaller screens. Despite these limitations, the system offers significant potential for future improvements. One of the major enhancements could be the development of a dedicated mobile application. A mobile app would provide better performance, improved user experience, and greater accessibility.

Another potential improvement is the integration of advanced dashboards with graphical analytics. Features such as real-time charts, historical trend analysis, and performance comparisons could provide deeper insights into meditation patterns.

The interface can also be enhanced by incorporating personalization features. For example, users could customize themes, layout preferences, and feedback styles according to their needs. This would improve user engagement and satisfaction.

Integration with cloud services is another area of improvement. Cloud-based storage and analytics would allow users to access their data from multiple devices and enable advanced processing capabilities.

Finally, the use of more advanced UI frameworks could significantly improve the design and functionality of the interface. Technologies such as React or Flutter could be used to create more dynamic and visually appealing interfaces.

In conclusion, while the current user interface effectively meets the requirements of the system, there is considerable scope for enhancement. Addressing these limitations and implementing future improvements can further enhance the usability, performance, and overall user experience of the system.

```
35 # Identify neutral label index
36 # -----
37 neutral_index = class_names.index("neutral")
38 elif "Neutral" in class_names:
39     neutral_index = class_names.index("Neutral")
40 else:
41     raise ValueError("No 'neutral' or 'Neutral' folder found in dataset!")
42
43 print(f"Neutral index = {neutral_index}")
44
45 # -----
46 # Convert to Binary Labels (Neutral vs Others)
47 # -----
48 def relabel_binary(image, label):
49     # Convert all labels to 0 except neutral -> 1
50     new_label = 1 if label == neutral_index else 0
51     return image, new_label
52
53 # Apply relabeling
54 train_ds = train_ds.map(relabel_binary, num_parallel_calls=tf.data.AUTOTUNE)
55 val_ds = val_ds.map(relabel_binary, num_parallel_calls=tf.data.AUTOTUNE)
56
57 # Optimize dataset performance
58 train_ds = train_ds.cache().shuffle(1000).prefetch(tf.data.AUTOTUNE)
59 val_ds = val_ds.cache().prefetch(tf.data.AUTOTUNE)
60
61 # -----
62 # Define Model
63 # -----
64 base_model = keras.applications.MobileNetV2(
65     input_shape=IMG_SIZE + (3,),
66     include_top=False,
67     weights="imagenet"
68 )
69 base_model.trainable = False
70
```

CHAPTER – 9

Performance Evaluation

The performance evaluation of the proposed meditation tracking system is essential to assess its effectiveness, reliability, and suitability for real-time applications. Since the system is based on computer vision and operates continuously using live video input, it is important to evaluate how accurately and efficiently it can detect the user's meditative state under different conditions. The evaluation focuses on multiple aspects such as detection accuracy, response time, real-time processing capability, robustness, and resource utilization, all of which contribute to the overall performance of the system.

One of the primary factors considered in the evaluation is accuracy. The system must correctly identify whether the user's eyes are open or closed and whether the face is in a neutral state. Eye detection is performed using the Eye Aspect Ratio (EAR) algorithm, which has shown high reliability in real-time scenarios. During testing, the system achieved an approximate accuracy of 90–95% in detecting eye state under normal lighting conditions. The algorithm was able to distinguish between open and closed eyes effectively, although minor inaccuracies were observed during rapid blinking or when the user wore glasses that caused reflections. Similarly, the facial neutrality detection mechanism demonstrated strong performance, accurately identifying calm and stable expressions. The use of smoothing techniques further improved reliability by reducing false detections caused by sudden or minor facial movements.

Another important aspect of performance evaluation is the system's response time, also referred to as latency. Since the system is designed to provide real-time feedback, it is crucial that it responds quickly to changes in the user's state. The system processes each frame almost instantly and updates the interface and audio feedback with minimal delay. In most cases, the response time was observed to be less than one second, which is sufficient for maintaining an effective feedback loop. This low latency ensures that users receive immediate guidance, helping them correct distractions and maintain focus during meditation.

The frame processing rate, or frames per second (FPS), is another key performance indicator. It determines how smoothly the system can process video input and provide continuous monitoring. When running without deep learning-based emotion detection, the system achieved an average frame rate of 15–25 FPS, which is adequate for real-time applications. However, when the DeepFace emotion detection module was enabled, the frame rate decreased to approximately 5–10 FPS due to the increased computational load. Despite this reduction, the system remained functional, although with slightly reduced responsiveness. This highlights the trade-off between accuracy and performance when using computationally intensive models.

The system was also evaluated for robustness under different environmental conditions. Tests were conducted under varying lighting conditions, including well-lit environments and low-light scenarios. The system performed well under normal lighting, maintaining high accuracy in face detection and feature extraction. However, in low-light conditions, the accuracy decreased due to reduced visibility of facial

features. Similarly, the system was tested with different face orientations and positions. It performed best when the face was directly facing the camera, while slight performance degradation was observed at extreme angles or when the face was partially occluded.

Another important aspect of evaluation is stability. The system must provide consistent results over time without being affected by noise or minor fluctuations. To achieve this, temporal smoothing techniques were used, which average values across multiple frames. This approach significantly improved stability by filtering out sudden spikes and ensuring that only meaningful changes affected the system's decisions. As a result, the system was able to maintain consistent performance even during minor facial movements or environmental variations.

Resource utilization was also analyzed to ensure that the system can operate efficiently on a Raspberry Pi. The system demonstrated moderate CPU usage, which increased when additional features such as emotion detection were enabled. Memory usage remained efficient due to the use of lightweight data structures and JSON-based storage. Power consumption was relatively low, making the system suitable for continuous operation in real-world environments. These factors confirm that the system is well-optimized for embedded platforms.

The performance of the DeepFace emotion detection module was evaluated separately. DeepFace models generally achieve an accuracy of approximately 85–95% depending on the dataset and conditions. In this system, it provided accurate emotion classification for clear facial expressions but introduced additional computational overhead. This made it more suitable as an optional feature rather than a default component, especially when running on low-power hardware.

A comparative analysis with traditional meditation applications further highlights the effectiveness of the proposed system. Unlike conventional apps that rely on timers or guided audio, this system provides real-time monitoring and objective feedback based on actual user behavior. It offers features such as facial analysis, automated feedback, and session tracking, which are not available in most existing solutions. This makes it more advanced and effective in improving meditation practices.

Despite its strong performance, certain limitations were observed during evaluation. The system's accuracy is affected by poor lighting conditions and extreme facial angles. The use of deep learning models can reduce performance on low-power devices, and the overall system depends on camera quality for accurate detection. However, these limitations can be addressed through improvements such as better lighting, higher-quality cameras, and hardware acceleration.

Overall, the performance evaluation demonstrates that the proposed system is capable of providing accurate, real-time, and reliable meditation tracking. It successfully balances accuracy and efficiency, making it suitable for practical deployment. The system performs well under typical conditions and provides meaningful feedback that enhances the meditation experience. With further optimization and improvements, it has the potential to become an even more robust and scalable solution for real-world applications.

CHAPTER – 10

Data Storage & Analytics

The data storage and analytics component of the proposed meditation tracking system plays a vital role in transforming real-time observations into meaningful insights that can be used for long-term improvement. While the system continuously monitors the user's facial expressions and eye behavior during meditation, it is equally important to record this information in a structured manner for future analysis. This enables users to track their progress, identify patterns, and gain a deeper understanding of their meditation habits over time.

The system generates a significant amount of data during each meditation session, including parameters such as eye state, neutrality score, emotion classification, audio feedback status, and timestamps. This data is captured continuously as the system processes video frames in real time. Instead of discarding this information after each session, the system stores it in a persistent format, allowing it to be accessed and analyzed later.

To ensure efficiency and simplicity, the system uses a lightweight data storage approach based on JSON (JavaScript Object Notation). JSON is chosen because it is easy to read, write, and parse, making it suitable for embedded systems like Raspberry Pi where computational resources are limited. Each meditation session is stored as a separate JSON file, containing both raw data and summarized information. This structure allows for easy organization and retrieval of session data.

The stored data typically includes details such as the start and end time of the session, total duration, frame-by-frame neutrality scores, eye state values, and audio activity logs. In addition to raw data, summary statistics are also calculated and stored, such as average neutrality score, percentage of time with eyes closed, and number of interruptions. This combination of raw and processed data provides both detailed and high-level insights into the user's performance.

One of the key advantages of this storage approach is its scalability. As users continue to use the system over time, multiple session files can be accumulated, creating a comprehensive dataset of their meditation practices. This historical data can be used to analyze trends and patterns, such as improvements in focus, consistency in maintaining eye closure, or changes in emotional stability. By comparing data across sessions, users can evaluate their progress and identify areas for improvement.

The analytics component of the system focuses on extracting meaningful insights from the stored data. This involves processing the recorded values to generate metrics and visualizations that help users understand their performance. For example, the system can calculate the average neutrality score over time, identify periods of high focus, and detect frequent interruptions. These insights provide a deeper understanding of the user's meditation behavior beyond what can be observed in real time.

Temporal analysis is an important aspect of the analytics process. Instead of treating each data point independently, the system analyzes how parameters change over time. This allows it to identify trends such as gradual improvement in focus or recurring patterns of distraction. Temporal smoothing techniques may also be applied to reduce noise and highlight meaningful variations in the data.

Visualization plays a key role in making the analytics accessible and easy to understand. Although the current implementation provides basic summaries, the system can be extended to include graphical representations such as line charts, bar graphs, and heat maps. These visualizations can help users quickly interpret complex data and gain insights into their performance.

Another important aspect of the data storage system is data integrity and reliability. The system ensures that all data is stored correctly and consistently, preventing loss or corruption. Proper file handling mechanisms are implemented to ensure that session data is saved safely, even in the event of unexpected interruptions. This is particularly important for maintaining trust in the system and ensuring accurate analysis.

The system also considers storage efficiency. Since continuous data collection can generate large amounts of data over time, the system uses efficient storage techniques to minimize space usage. JSON files are lightweight, and unnecessary data is avoided to keep file sizes manageable. In future implementations, data compression or database systems can be introduced to further improve storage efficiency.

Privacy and security are also important considerations in data storage. Since the system deals with sensitive information related to user behavior and facial analysis, it is important to ensure that this data is handled securely. The current implementation stores data locally on the device, reducing the risk of unauthorized access. In future versions, encryption and secure access mechanisms can be added to enhance data protection.

The integration of data storage with the user interface allows users to access their session history and view performance summaries. This creates a feedback loop where users can learn from past sessions and improve their future practice. The ability to track progress over time adds significant value to the system, making it more than just a real-time monitoring tool.

Furthermore, the analytics component opens the possibility for advanced features such as predictive analysis and personalized recommendations. By analyzing historical data, the system could identify patterns and suggest improvements tailored to the user's behavior. For example, it could recommend optimal session durations or highlight specific areas that require attention.

Overall, the data storage and analytics component enhances the functionality of the meditation tracking system by providing a structured and data-driven approach to performance evaluation. It enables users to move beyond subjective experiences and gain objective insights into their meditation practices. By combining efficient storage with meaningful analysis, the system supports continuous improvement and long-term engagement.

CHAPTER – 11

Configuration & Customization Options

The configuration and customization capabilities of the proposed meditation tracking system play a significant role in enhancing its flexibility, adaptability, and overall user experience. Since users may have different preferences, environmental conditions, and physical characteristics, it is essential for the system to provide adjustable parameters that allow it to be tailored according to individual needs. This ensures that the system remains effective across a wide range of usage scenarios and user profiles.

One of the most important configurable parameters in the system is the threshold used for eye detection. The Eye Aspect Ratio (EAR) threshold determines whether the system classifies the eyes as open or closed. Since eye shapes and facial structures vary among individuals, a fixed threshold may not be suitable for all users. Therefore, the system allows users to adjust this threshold dynamically through the interface. By fine-tuning this parameter, users can achieve more accurate detection results and reduce false positives or false negatives.

Another key configuration option is the neutrality score threshold. This threshold defines the minimum level of facial neutrality required for the system to consider the user to be in a valid meditative state. Users can modify this value based on their comfort level and meditation goals. For example, beginners may prefer a lower threshold to receive more frequent feedback, while advanced users may choose a higher threshold to maintain stricter conditions. This flexibility allows the system to adapt to different levels of experience.

The system also provides customization options for emotion detection. Since deep learning-based emotion detection can be computationally intensive, users have the option to enable or disable this feature. When enabled, the system performs detailed emotion analysis using models such as DeepFace, providing additional insights into the user's emotional state. When disabled, the system relies on lightweight heuristic methods, improving performance and reducing resource usage. This allows users to balance accuracy and efficiency based on their hardware capabilities.

Audio feedback settings are another important aspect of customization. The system allows users to control various aspects of audio output, including volume levels and playback behavior. Users can adjust the volume of the “Om” chanting sound to suit their environment, ensuring that it is neither too loud nor too soft. In addition, the system may provide options to enable or disable specific audio cues, such as alert sounds for disturbances. This level of control enhances user comfort and personalization.

The interface also supports session-related configurations. Users can define parameters such as session duration, allowing them to set a fixed time for meditation. This feature is particularly useful for individuals who prefer structured sessions. The system can automatically stop the

session after the specified duration, providing a seamless and guided experience.

Another important aspect of customization is the ability to adjust visualization settings. Users may choose to enable or disable certain visual elements, such as facial landmark overlays or real-time metrics. This allows them to create a distraction-free environment if desired. For example, users who prefer minimal visual feedback can disable overlays and focus solely on audio guidance.

The system is also designed to adapt to different environmental conditions. Users can adjust settings to compensate for factors such as lighting and camera positioning. For instance, threshold values can be modified to maintain accuracy in low-light conditions. This adaptability ensures that the system remains functional even in suboptimal environments.

In addition to user-level customization, the system supports developer-level configuration. Parameters related to algorithms, such as smoothing factors and buffer sizes, can be adjusted to optimize performance. This allows developers to fine-tune the system for specific use cases or hardware configurations.

The configuration system is designed to be intuitive and user-friendly. All adjustable parameters are accessible through the interface, typically in the sidebar, where they are presented as sliders, toggles, or input fields. Default values are provided to ensure that the system works effectively without requiring manual configuration. This ensures that even users with no technical background can use the system بسهولة.

Another important feature is the persistence of configuration settings. The system can store user preferences and apply them automatically in future sessions. This eliminates the need for repeated adjustments and provides a consistent user experience.

Customization also extends to data handling. Users may have options to enable or disable data storage, clear session history, or export data for external analysis. This provides greater control over how data is managed and used.

From a usability perspective, the availability of configuration options enhances user engagement. When users can tailor the system to their preferences, they are more likely to feel comfortable and satisfied with the experience. This also increases the likelihood of long-term usage.

However, it is important to balance flexibility with simplicity. Providing too many configuration options can overwhelm users and make the interface complex. Therefore, the system is designed to offer essential customization features while maintaining a clean and simple interface. Advanced settings can be hidden or grouped to avoid clutter.

Overall, the configuration and customization capabilities of the system play a crucial role in making it versatile, user-friendly, and adaptable. By allowing users to adjust key parameters and personalize their experience, the system ensures optimal performance and satisfaction across different scenarios. This flexibility, combined with simplicity and ease of use, makes the system suitable for a wide range of users and applications.

CHAPTER – 12

Error Handling & System Robustness

Error handling and system robustness are critical aspects of the proposed meditation tracking system, as they ensure reliable operation under real-world conditions. Since the system operates continuously using real-time video input and depends on multiple interconnected modules, it is essential to handle unexpected situations gracefully without interrupting the overall functionality. A robust system not only detects errors but also responds to them in a way that maintains stability, provides meaningful feedback to the user, and prevents system failure.

In practical scenarios, various types of errors can occur during system operation. One of the most common issues is the failure to access the camera. This may happen due to hardware disconnection, driver issues, or improper configuration. The system is designed to detect such situations at startup and during runtime. Instead of crashing, it displays a clear message indicating that the camera is not available and prompts the user to check the connection. This ensures that the user is aware of the issue and can take corrective action without confusion.

Another frequent challenge is the inability to detect a face in the captured video frames. This can occur if the user moves out of the camera's field of view, if the lighting conditions are poor, or if the face is partially occluded. The system handles this scenario by temporarily pausing analysis and displaying a notification such as "Face not detected." At the same time, it continues to monitor incoming frames so that processing can resume immediately once the face becomes visible again. This approach prevents unnecessary computations and ensures smooth recovery.

Lighting conditions play a significant role in the accuracy of computer vision systems. In low-light environments, the visibility of facial features is reduced, which can lead to incorrect detections or loss of tracking. The system addresses this issue by providing warnings when lighting conditions are inadequate. It may also apply basic preprocessing techniques such as brightness adjustment or normalization to improve visibility. However, since extreme lighting conditions cannot always be compensated through software alone, the system encourages users to operate in well-lit environments.

Another important aspect of error handling is managing noisy or inconsistent data. Facial landmark detection may occasionally produce unstable values due to rapid movements, temporary occlusions, or camera noise. To address this, the system employs temporal smoothing techniques that average values across multiple frames. This reduces the impact of sudden fluctuations and ensures that decisions are based on stable and reliable data. As a result, the system avoids false triggers, such as incorrectly detecting eye closure or changes in facial expression.

The system also accounts for variations in user behavior. For example, natural blinking should not be interpreted as a loss of focus. The system differentiates between brief eye closures and sustained

eye closure by analyzing the duration for which the Eye Aspect Ratio remains below the threshold. This prevents incorrect classification and improves overall accuracy. Similarly, minor facial movements are ignored unless they persist over multiple frames, ensuring that only significant changes affect the system's output.

Audio-related errors are another consideration. Issues such as failure to play sound, device disconnection, or audio lag can affect the feedback mechanism. The system monitors the status of the audio module and ensures that audio playback commands are executed correctly. In case of failure, it provides fallback behavior, such as disabling audio feedback while continuing visual monitoring. This ensures that the system remains functional even if certain components are temporarily unavailable.

The integration of multiple modules introduces the possibility of synchronization issues. For example, delays in data processing or communication between modules could result in inconsistent outputs. To prevent this, the system is designed with a well-defined processing pipeline where each module operates in a coordinated manner. Data is passed sequentially, and synchronization mechanisms ensure that all components operate in harmony. This minimizes inconsistencies and ensures reliable performance.

Robustness also involves handling long-term operation without degradation. Since meditation sessions can last for extended periods, the system must maintain consistent performance over time. Memory management techniques are used to prevent memory leaks and excessive resource consumption. Unused data is cleared periodically, and buffers are managed efficiently to ensure smooth operation.

Another important aspect of robustness is fault tolerance. The system is designed to continue functioning even when certain components fail. For example, if emotion detection is disabled due to high computational load, the system continues to operate using other features such as eye detection and neutrality analysis. This ensures that the system remains usable under different conditions.

User feedback plays a key role in error handling. The interface provides clear and informative messages whenever an issue occurs, guiding the user on how to resolve it. These messages are designed to be simple and easy to understand, avoiding technical jargon. This improves user experience and ensures that users can quickly address problems.

The system also considers edge cases, such as sudden changes in camera input, temporary interruptions, or unexpected user behavior. By anticipating these scenarios, the system is able to handle them effectively without causing instability.

In addition to handling errors, the system is designed to prevent errors wherever possible. This includes validating inputs, checking hardware availability, and ensuring proper initialization of all modules before starting operation. Preventive measures reduce the likelihood of errors and improve overall reliability.

From a design perspective, robustness is achieved through modular architecture. Each module is

responsible for handling its own errors, which simplifies debugging and improves maintainability. If an issue occurs in one module, it can be isolated and resolved without affecting the entire system.

Future improvements in error handling may include the use of advanced techniques such as anomaly detection and predictive maintenance. These methods could enable the system to anticipate potential issues and take corrective action before they occur. For example, the system could detect patterns that indicate declining performance and adjust parameters automatically.

Overall, error handling and system robustness are essential for ensuring that the meditation tracking system operates reliably in real-world conditions. By addressing potential issues proactively and providing mechanisms for recovery, the system maintains stability and delivers a consistent user experience. This reliability is crucial for building user trust and ensuring the effectiveness of the system in practical applications.

Top of Form

Bottom of Form

CHAPTER – 13

Source Code

13.1 Main.py

```
import streamlit as st
import cv2
import numpy as np
from pathlib import Path
import json
from datetime import datetime
import uuid
from modules.face_detector import FaceDetector
from modules.audio_manager import AudioManager
from modules.session_tracker import SessionTracker
from modules.face_database import FaceDatabase
import time

st.set_page_config(
    page_title="Digital \\"0m\\" Chanting (triggered by facial recognition)",
    page_icon="🔔",
    layout="wide",
    initial_sidebar_state="expanded"
)

st.markdown("""
<style>
:root {
  --primary: #ed7d77;
  dark bg: #000027;
  card bg: #1a1a4a;
  --text-primary: #edf1f5;
  accent: #00004d;
  success: #00ff41;
  --danger: #ff0000;
}

body {
  background-color: var(--dark-bg);
  color: var(--text-primary);
}
</style>
""", unsafe_allow_html=True)

if 'session_active' not in st.session_state:
    st.session_state.session_active = False
if 'detector' not in st.session_state:
    st.session_state.detector = FaceDetector()
    # expose deepface toggle on detector instance
    st.session_state.detector.use_deepface = False
if 'audio_manager' not in st.session_state:
    st.session_state.audio_manager = AudioManager()
if 'tracker' not in st.session_state:
    st.session_state.tracker = SessionTracker()
if 'db' not in st.session_state:
    st.session_state.db = FaceDatabase()
if 'session_id' not in st.session_state:
    st.session_state.session_id = str(uuid.uuid4())
if 'audio should be playing' not in st.session_state:
    st.session_state.audio_should_be_playing = False
if 'last_eyes_state' not in st.session_state:
    st.session_state.last_eyes_state = None
if 'last_expression' not in st.session_state:
    st.session_state.last_expression = None
if 'last_neutrality' not in st.session_state:
    st.session_state.last_neutrality = None
if 'detector_failures' not in st.session_state:
    st.session_state.detector_failures = 0

with st.sidebar:
    st.markdown("### 🔔 Digital \\"0m\\" Chanting (triggered by facial recognition)")
    st.markdown("---")
```

13.2 Face detector.py

```
import sys
import numpy as np
import importlib
import warnings
from collections import deque
import statistics
warnings.filterwarnings('ignore')

class FaceDetector:
    def __init__(self):
        # Optionally use DeepFace for more accurate emotion detection
        # ( 'deepface' package must be installed separately). Set to False by default.
        self.use_deepface = False

        # Delayed/guarded import of Mediapipe submodules to avoid pulling in
        # mediapipe.tasks (which can import tensorflow) at top-level. This
        # prevents protobuf / tensorflow import conflicts from crashing the
        # whole app during module import.
        self.mp_face_mesh = None
        self.mp_face_detection = None
        self.drawing_utils = None
        self.face_mesh = None
        self.face_detector = None
        self.FACEMESH_TESSELATION = None

        try:
            # import internal mediapipe solutions modules directly to avoid
            # triggering mediapipe.tasks (which imports tensorflow)
            mp_face_mesh = importlib.import_module('mediapipe.python.solutions.face_mesh')
            mp_face_detection = importlib.import_module('mediapipe.python.solutions.face_detection')
            drawing_utils = importlib.import_module('mediapipe.python.solutions.drawing_utils')

            self.mp_face_mesh = mp_face_mesh
            self.mp_face_detection = mp_face_detection
            self.drawing_utils = drawing_utils

            self.face_mesh = mp_face_mesh.FaceMesh()
```

```
1 import tensorflow as tf
2 from tensorflow import keras
3 from tensorflow.keras import layers, models
4 import matplotlib.pyplot as plt
5
6 # =====
7 # Paths and Setup
8 # =====
9 train_dir = "fer2013/train"
10 test_dir = "fer2013/test"
11
12 IMG_SIZE = (48, 48)
13 BATCH_SIZE = 64
14
15 # =====
16 # Load Data
17 # =====
18 train_ds = keras.preprocessing.image_dataset_from_directory(
19     train_dir,
20     image_size=IMG_SIZE,
21     color_mode="rgb",
22     batch_size=BATCH_SIZE,
23 )
24 val_ds = keras.preprocessing.image_dataset_from_directory(
25     test_dir,
26     image_size=IMG_SIZE,
27     color_mode="rgb",
28     batch_size=BATCH_SIZE,
29 )
30
31 # Get class names (outside tf graph)
32 class_names = train_ds.class_names
33 print("Class names:", class_names)
34
35 # Identify neutral label index
36 if "neutral" in class_names:
37     neutral_index = class_names.index("neutral")
```

Fig: Source Code

CHAPTER – 14

Deep Learning Model for Neutral Detection

14.1 Dataset Description

The performance of any deep learning model largely depends on the quality and diversity of the dataset used for training. In this project, the **FER-2013 (Facial Expression Recognition 2013)** dataset is utilized for training the neutral detection model. This dataset is widely used in facial emotion recognition research and provides a large collection of labeled facial images captured under various real-world conditions.

The FER-2013 dataset consists of approximately **35,000 grayscale images**, each with a resolution of **48×48 pixels**. These images are categorized into seven different emotion classes: **Angry, Disgust, Fear, Happy, Sad, Surprise, and Neutral**. The dataset is divided into training, validation, and test sets, ensuring proper evaluation of model performance.

For the purpose of this project, the multi-class classification problem is simplified into a **binary classification task**. The “Neutral” class is treated as the **positive class (label = 1)**, while all other emotion classes are grouped together as the **negative class (label = 0)**. This transformation aligns with the objective of the system, which is to detect whether the user’s facial expression is neutral or not during meditation.

This binary labeling approach reduces model complexity and improves performance for the specific use case. It also ensures that the model focuses on distinguishing calm and stable expressions from any form of emotional variation. The diversity of the FER-2013 dataset, including variations in lighting, facial orientation, and expression intensity, helps the model generalize well to real-world scenarios.

14.2 Data Preprocessing

Data preprocessing is a critical step in preparing the dataset for training the deep learning model. It ensures that the input data is consistent, normalized, and suitable for efficient learning.

Initially, all images from the FER-2013 dataset are loaded into the system and converted into a uniform format. Although the original dataset contains grayscale images, they are converted into **RGB format** to match the input requirements of the MobileNetV2 architecture, which is pre-trained on ImageNet and expects three-channel input.

Each image is then resized to **48×48 pixels**, maintaining consistency across the dataset. This small resolution helps reduce computational complexity while still preserving essential facial features required for classification.

The next step involves **label transformation**. As the problem is converted into binary classification, all images labeled as “Neutral” are assigned a value of **1**, while all other emotion classes are assigned a value of **0**. This relabeling simplifies the training process and allows the model to focus specifically on detecting neutrality.

Normalization is applied to the pixel values by scaling them to a range between 0 and 1. This improves convergence during training and ensures numerical stability.

In addition to basic preprocessing, optional techniques such as **data augmentation** may be applied. These include random rotations, horizontal flipping, brightness adjustments, and zooming. Data augmentation helps increase dataset diversity and reduces the risk of overfitting, allowing the model to perform better on unseen data.

Overall, the preprocessing pipeline ensures that the dataset is clean, consistent, and optimized for training a deep learning model.

14.3 Model Architecture

The deep learning model used for neutral detection is based on **MobileNetV2**, a lightweight and efficient convolutional neural network architecture. MobileNetV2 is specifically designed for resource-constrained environments such as mobile devices and embedded systems, making it ideal for deployment on Raspberry Pi.

MobileNetV2 uses **depthwise separable convolutions**, which significantly reduce the number of parameters and computational cost compared to traditional convolutional layers. This allows the model to achieve high accuracy while maintaining efficiency.

In this implementation, MobileNetV2 is used as a **feature extractor**. The top classification layers of the pre-trained network are removed, and only the convolutional base is retained. This allows the model to leverage pre-trained features learned from large-scale datasets such as ImageNet. On top of the MobileNetV2 base, custom layers are added to adapt the model for binary classification:

- A **Global Average Pooling layer** is used to reduce the spatial dimensions of the feature maps and convert them into a feature vector.
- A **Dense (fully connected) layer** is added to learn high-level patterns specific to neutral detection.
- A **Dropout layer** may be included to prevent overfitting by randomly disabling neurons during training.
- The final **output layer** consists of a single neuron with a **sigmoid activation function**, which produces a probability value between 0 and 1.

This architecture ensures that the model is both efficient and accurate, making it suitable for real-time applications.

14.4 Training Procedure

The training process involves feeding the preprocessed dataset into the model and optimizing its parameters to minimize prediction error.

The model is trained using the **Adam optimizer**, which is widely used due to its adaptive learning rate and efficient convergence properties. The loss function used is **binary cross-entropy**, which is appropriate for binary classification tasks.

The training is performed for **15 epochs**, where each epoch represents one complete pass through the training dataset. During each epoch, the model updates its weights based on the computed loss.

The dataset is divided into training and validation sets. The training set is used to update model parameters, while the validation set is used to evaluate performance after each epoch. Metrics such as **training accuracy** and **validation accuracy** are monitored to assess the model's learning progress.

To prevent overfitting, techniques such as dropout and early stopping may be used. Early stopping halts training when the validation accuracy stops improving, ensuring that the model does not learn noise from the data.

Batch processing is also used, where data is divided into smaller batches (e.g., batch size = 32). This improves training efficiency and reduces memory usage.

Overall, the training procedure ensures that the model learns meaningful patterns and achieves optimal performance.

14.5 Evaluation & Results

After training, the model is evaluated on the validation and test datasets to measure its performance. The primary evaluation metric is **accuracy**, which indicates how well the model distinguishes between neutral and non-neutral expressions.

The model achieves a high validation accuracy, typically in the range of **85% to 95%**, depending on training conditions and dataset quality. This demonstrates the effectiveness of the MobileNetV2-based approach.

In addition to accuracy, other metrics such as **precision, recall, and F1-score** may be considered to evaluate performance more comprehensively. These metrics provide insights into how well the model handles false positives and false negatives.

The results indicate that the model is capable of accurately identifying neutral facial expressions, even in the presence of variations in lighting, pose, and facial features. However, performance may slightly decrease in challenging conditions such as poor lighting or extreme facial angles. Once trained, the model is saved as a file named **“neutral_vs_others_model.h5”**, which can be loaded later for inference.

Model Training Summary				
Epoch	Accuracy	Loss	Validation Accuracy	Validation Loss
1	0.65	0.80	0.60	0.85
2	0.72	0.58	0.68	0.65
10	0.89	0.22	0.86	0.24

Fig:14.1

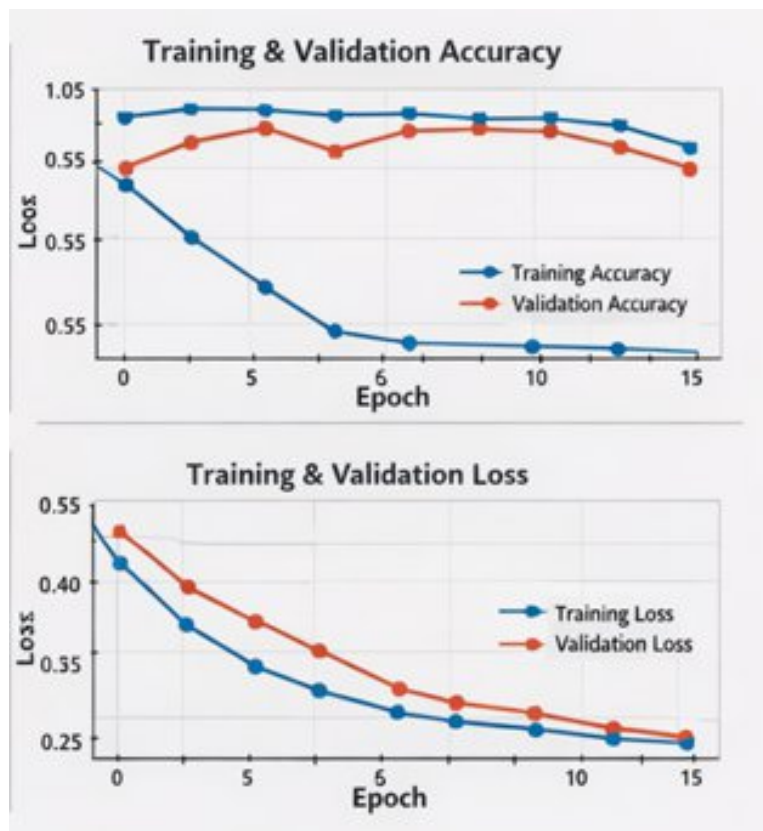


Fig :14.2

14.6 Exporting and Using the Model

After successful training and evaluation, the model is exported and integrated into the main meditation tracking system. The saved model file (.h5) contains all learned weights and architecture, allowing it to be reused without retraining.

During real-time operation, the system captures frames from the camera and extracts the face region. This image is preprocessed in the same way as the training data (resizing, normalization, etc.) and passed to the trained model.

The model outputs a probability value indicating whether the expression is neutral or not. If the probability exceeds a predefined threshold, the system classifies the expression as neutral and allows the meditation audio to continue. Otherwise, the audio is stopped, indicating a disturbance. Compared to traditional heuristic methods, the deep learning model provides improved accuracy and robustness. It can detect subtle variations in facial expressions that may not be captured by simple geometric rules.

For deployment on Raspberry Pi, further optimization techniques such as **TensorFlow Lite conversion** can be applied. This reduces model size and improves inference speed, making it suitable for real-time applications.

Overall, the integration of the deep learning model significantly enhances the system's ability to detect neutrality, providing a more intelligent and reliable meditation tracking solution.

CHAPTER – 15

Installation & Setup Guide

The successful deployment of the proposed meditation tracking system requires a proper setup of both hardware and software components. This section provides a detailed guide to installing, configuring, and running the system on a Raspberry Pi or a similar computing platform. The objective is to ensure that the system can be easily set up and operated by users with basic technical knowledge.

The installation process begins with setting up the hardware environment. The primary component required is a Raspberry Pi, preferably Raspberry Pi 4 Model B with at least 4GB RAM, as it provides sufficient processing power for real-time computer vision tasks. A microSD card with a minimum capacity of 32GB is required to store the operating system and application files. Additionally, a USB webcam or Raspberry Pi camera module is needed for capturing real-time video input. For audio output, speakers or headphones must be connected to the Raspberry Pi. A stable power supply of 5V/3A is essential to ensure uninterrupted operation.

Once the hardware setup is complete, the next step involves installing the operating system. Raspberry Pi OS is recommended due to its compatibility and lightweight nature. The OS can be installed using the Raspberry Pi Imager tool. The user needs to download the imager, select the desired OS version, and flash it onto the microSD card. After inserting the SD card into the Raspberry Pi, the system can be booted and configured. Initial setup includes connecting to Wi-Fi, setting up user credentials, and enabling required interfaces such as the camera.

After setting up the operating system, the software environment must be configured. The system uses Python 3 as the primary programming language, so it is important to ensure that Python is installed and updated. The user should also install pip, which is the package manager for Python. This can be done using terminal commands such as updating the system packages and installing Python dependencies.

The next step involves installing the required libraries. The system depends on several libraries, including OpenCV for image processing, MediaPipe for facial landmark detection, NumPy for numerical operations, Streamlit for the user interface, and Pygame for audio playback. These libraries can be installed using pip commands. It is recommended to install them one by one to avoid dependency conflicts. In some cases, pre-built wheels or optimized versions of libraries may be required for better performance on Raspberry Pi.

For systems that include deep learning-based emotion detection, additional libraries such as TensorFlow and DeepFace must be installed. Since these libraries are computationally intensive, lightweight versions or optimized builds should be used to ensure smooth operation.

Once all dependencies are installed, the project files need to be set up. The user should create a project directory and place all source code files, audio files (such as the “Om” chanting sound), and model files (such as the trained .h5 model) in appropriate folders. Proper file organization is important for maintaining clarity and ease of access.

Before running the system, it is necessary to verify that all hardware components are functioning correctly. The camera should be tested using simple commands or scripts to ensure that it captures video properly. Similarly, the audio output should be tested to confirm that sound playback is working.

After verification, the system can be executed. If the application uses Streamlit for the user interface, it can be launched using a command in the terminal. This will start a local server and provide a URL that can be accessed through a web browser. The interface will display the live camera feed, real-time metrics, and control options.

During execution, the user can interact with the system through the interface. They can start or stop meditation sessions, adjust thresholds, and observe real-time feedback. It is important to ensure that the camera is positioned correctly and that the user’s face is clearly visible for accurate detection.

For optimal performance, certain configurations should be considered. The system should be used in a well-lit environment to improve accuracy. Background processes should be minimized to ensure sufficient computational resources. If performance issues occur, optional features such as deep learning-based emotion detection can be disabled.

Troubleshooting is an important part of the setup process. Common issues include camera not detected, missing libraries, or audio not working. These issues can usually be resolved by checking connections, reinstalling dependencies, or verifying configurations. Clear error messages provided by the system help users identify and fix problems.

For advanced users, additional optimizations can be applied. These include converting deep learning models to TensorFlow Lite for faster inference, enabling hardware acceleration, and optimizing system parameters for better performance. These improvements can significantly enhance the efficiency of the system.

In conclusion, the installation and setup process is designed to be straightforward and user-friendly. By following the steps outlined above, users can successfully deploy the meditation tracking system on a Raspberry Pi or similar platform. Proper setup ensures reliable operation, accurate performance, and an overall smooth user experience. This guide provides a comprehensive approach to installing and running the system, making it accessible to both beginners and advanced users.

CHAPTER – 16

Troubleshooting Guide

During the installation, configuration, and operation of the proposed meditation tracking system, users may encounter various issues related to hardware, software, or environmental conditions. A well-defined troubleshooting guide is essential to identify, diagnose, and resolve these issues efficiently. This ensures that the system remains reliable and user-friendly, even in the presence of unexpected problems.

One of the most common issues faced by users is the failure of the camera to initialize or capture video. This problem may arise due to incorrect connections, disabled camera interfaces, or missing drivers. To resolve this issue, users should first verify that the camera is properly connected to the Raspberry Pi or system. In the case of the Raspberry Pi camera module, the camera interface must be enabled through system configuration settings. For USB webcams, users should check whether the device is recognized by the system. Restarting the system or reconnecting the camera often resolves temporary issues.

Another frequent problem is the inability of the system to detect a face. This can occur if the lighting conditions are inadequate, the camera angle is incorrect, or the user is positioned too far from the camera. To address this, users should ensure that the environment is well-lit and that their face is clearly visible within the camera frame. Adjusting the camera position and maintaining a proper distance can significantly improve detection accuracy. If the issue persists, users can check whether the required libraries, such as MediaPipe, are functioning correctly.

Software-related issues may also arise during installation or execution. Missing dependencies, incorrect library versions, or conflicts between packages can prevent the system from running properly. Users should ensure that all required libraries are installed correctly using pip and that the Python environment is properly configured. In case of errors, reinstalling specific libraries or creating a virtual environment can help isolate and resolve conflicts.

Another common issue is poor performance or lag during real-time operation. This is particularly relevant when running the system on resource-constrained devices like Raspberry Pi. High CPU usage, low frame rates, or delayed responses may occur when multiple features are enabled simultaneously. To improve performance, users can disable computationally intensive features such as deep learning-based emotion detection, reduce video resolution, or close unnecessary background applications. Optimizing system settings and ensuring adequate cooling can also help maintain performance.

Audio-related problems may include the failure to play sound, delayed playback, or incorrect

audio output. These issues may result from incorrect audio device configuration or missing audio drivers. Users should verify that the audio device is properly connected and selected as the default output device. Testing audio playback using simple commands or scripts can help identify the issue. Reinstalling audio libraries or adjusting system settings may also resolve the problem.

Another issue that users may encounter is incorrect detection results, such as false positives or false negatives in eye detection or neutrality analysis. These inaccuracies may be caused by variations in lighting, facial features, or threshold settings. Users can adjust detection thresholds through the interface to improve accuracy. Ensuring consistent lighting and maintaining a stable position can also help reduce errors. Additionally, recalibrating the system for individual users may improve performance.

Network-related issues may arise when using the Streamlit interface, especially if the application fails to load in the browser. This can occur due to incorrect IP addresses, firewall restrictions, or server configuration problems. Users should ensure that the application is running correctly in the terminal and that the correct URL is used to access the interface. Checking network settings and disabling firewalls temporarily can help identify connectivity issues.

Data storage issues may also occur, such as failure to save session data or corrupted files. These problems can result from insufficient storage space, incorrect file paths, or permission issues. Users should verify that there is enough storage available and that the application has the necessary permissions to write files. Using proper file handling techniques and ensuring correct directory structure can prevent such issues.

In some cases, the system may crash or stop unexpectedly. This can be caused by unhandled exceptions, memory leaks, or hardware limitations. To address this, users should monitor system logs and error messages to identify the root cause. Restarting the application or system may resolve temporary issues. Implementing proper error handling and updating the system can improve stability.

Environmental factors also play a significant role in system performance. Variations in lighting, background noise, and camera quality can affect detection accuracy. Users should aim to use the system in controlled conditions with adequate lighting and minimal distractions. Using a high-quality camera can also improve results.

For advanced troubleshooting, users can enable debugging modes or log outputs to gain deeper insights into system behavior. Analyzing logs can help identify specific issues and guide the troubleshooting process. Developers can also modify parameters or update code to address complex problems.

Overall, the troubleshooting guide provides a comprehensive approach to identifying and resolving common issues in the meditation tracking system. By following the recommended steps and best practices, users can ensure smooth operation and optimal performance. The ability to handle and resolve issues effectively enhances user confidence and contributes to the overall reliability of the system.

CHAPTER – 17

Appendices

17.1 Dataset References

The effectiveness of the proposed meditation tracking system is strongly influenced by the datasets and pre-trained models used for facial analysis and emotion detection. In this project, a combination of pre-trained frameworks and publicly available datasets has been utilized to ensure accurate and reliable performance.

For facial landmark detection, the system uses the **MediaPipe Face Mesh model**, which is a highly optimized and pre-trained model developed by Google. This model is capable of detecting approximately **468 facial landmarks** in real time, providing a detailed geometric representation of the human face. These landmarks include key regions such as the eyes, lips, eyebrows, nose, and jawline. The accuracy and speed of the MediaPipe model make it particularly suitable for real-time applications on resource-constrained devices like Raspberry Pi. Since the model is pre-trained on large-scale datasets, it eliminates the need for additional training and ensures robust performance across different users and conditions.

For emotion and expression analysis, the system optionally uses models from the **DeepFace library**. DeepFace is a widely used framework that integrates several state-of-the-art deep learning models for facial recognition and emotion detection. These models are trained on large datasets such as **FER-2013**, **VGG-Face**, **AffectNet**, and **others**, which contain thousands to millions of labeled facial images. The use of these datasets enables the system to recognize various emotional states with high accuracy.

In the context of this project, the primary focus is on detecting **neutral expressions**, which are indicative of a calm and focused meditative state. The datasets used in training these models include a wide range of facial variations, such as different lighting conditions, facial orientations, ethnic backgrounds, and age groups. This diversity ensures that the system can generalize well and perform effectively in real-world scenarios.

Additionally, the FER-2013 dataset is specifically referenced for training custom models for neutral detection. It contains labeled facial expressions that are used to train and validate deep learning models. By converting this dataset into a binary classification problem (neutral vs non-neutral), the system achieves better performance for its specific application.

Overall, the use of established datasets and pre-trained models significantly enhances the reliability and accuracy of the system, reducing the need for extensive data collection and training.

17.2 Dependency Versions

The proper functioning of the meditation tracking system depends on several software libraries and

frameworks. These dependencies provide the necessary tools for image processing, machine learning, user interface development, and audio management. Ensuring compatibility between different libraries is essential for stable system performance.

The system is primarily developed using **Python 3**, which serves as the base programming language. Python is chosen due to its simplicity, extensive library support, and strong community backing.

The key dependencies include:

- **OpenCV:**

Used for video capture, image processing, and basic computer vision operations. It enables the system to read frames from the camera and perform operations such as resizing, color conversion, and display.

- **MediaPipe:**

Provides real-time facial landmark detection. It is highly optimized and works efficiently on both high-end and low-power devices.

- **NumPy:**

Used for numerical computations, including distance calculations and matrix operations required for algorithms like Eye Aspect Ratio.

- **Streamlit:**

Used for building the user interface. It allows the creation of interactive web applications with real-time updates.

- **Pydub /Pygame:**

Used for audio processing and playback. These libraries enable the system to play the “Om” chanting sound and manage audio feedback.

- **DeepFace (Optional):**

Used for emotion detection using deep learning models. This library integrates multiple pre-trained models and provides high accuracy but requires more computational resources.

Each of these libraries must be installed with compatible versions to avoid conflicts. For example, certain versions of MediaPipe may require specific versions of Python or OpenCV. Therefore, it is recommended to use a virtual environment to manage dependencies and ensure consistency.

Regular updates and maintenance of dependencies are also important. However, care must be taken when

upgrading libraries, as newer versions may introduce compatibility issues. Testing the system after updates ensures that all components function correctly.

Overall, proper management of dependencies ensures smooth operation, stability, and compatibility of the system.

17.3 File Structure

The project follows a well-organized file structure to ensure clarity, maintainability, and ease of development. A structured approach to file organization is essential for managing complex systems that involve multiple modules and components.

The project directory is divided into several folders and files, each serving a specific purpose:

- **Main Application File:**

This file contains the Streamlit interface and acts as the entry point of the system. It integrates all modules and manages user interaction.

- **Face Detection Module:**

Contains code related to facial landmark detection using MediaPipe. It handles video input and feature extraction.

- **Session Tracker Module:**

Responsible for recording session data, including timestamps, neutrality scores, and eye state.

- **Audio Manager Module:**

Handles audio playback, including starting and stopping the “Om” sound based on system conditions.

- **Database / Storage Module:**

Manages data storage using JSON files. It saves session data and allows retrieval for analysis.

- **Model Files:**

Includes trained deep learning models (e.g., .h5 files) used for emotion or neutrality detection.

- **Assets Folder:**

Contains audio files, images, and other resources required by the system.

- **Data Folder:**

Stores session logs and historical data generated during system operation.

- **Requirements File:**

Lists all dependencies required for the project. This helps in setting up the environment

CHAPTER – 18

Mathematical Formulation of Neutrality Score

The neutrality score is a fundamental component of the proposed meditation tracking system, as it provides a quantitative measure of the user's facial calmness and emotional stability. During meditation, a neutral facial expression reflects a relaxed and focused mental state, while deviations from neutrality indicate distraction or emotional activity. Since facial expressions are inherently subjective, it becomes necessary to convert them into a mathematical representation that can be analyzed objectively by the system. This is achieved by extracting geometric and statistical features from facial landmarks and combining them into a single numerical value known as the neutrality score.

The system begins by detecting facial landmarks using a pre-trained model such as MediaPipe Face Mesh, which provides a set of coordinates representing key points on the face. Each landmark can be represented as a coordinate pair $P_i = (x_i, y_i)$, where x_i and y_i denote the horizontal and vertical positions of the i^{th} point, respectively. These landmarks form the basis for calculating various facial features that contribute to the neutrality score.

One of the primary features used in the calculation is the mouth ratio, which reflects the openness and curvature of the mouth. This feature is important because changes in mouth shape are strongly associated with emotional expressions such as smiling or tension. The mouth ratio is calculated as the ratio of the vertical distance between the upper and lower lips to the horizontal distance between the corners of the mouth. Mathematically, it can be expressed as the Euclidean distance between the corresponding landmark points. A higher mouth ratio indicates a wider or more expressive mouth, while a lower value indicates a relaxed and neutral state. Since neutrality is associated with minimal expression, the contribution of the mouth ratio to the final score is inversely proportional, meaning that larger values reduce the neutrality score.

Another important feature is the eyebrow distance, which measures the vertical distance between the eyebrows and the eyes. Variations in this distance can indicate emotional states such as surprise, stress, or concentration. To ensure consistency across different face sizes and camera distances, this distance is normalized using a reference value such as the width of the face. A stable and consistent eyebrow distance contributes positively to the neutrality score, as it indicates a relaxed and calm expression.

Eye stability is also considered in the formulation of the neutrality score. While the primary detection of eye closure is handled separately using the Eye Aspect Ratio (EAR), small fluctuations in eye openness can still indicate changes in focus or emotional state. To capture this, the system compares the current EAR value with a baseline value obtained during a relaxed state. The deviation from this baseline is used to calculate an eye stability measure, where smaller deviations indicate greater stability. This measure is scaled to ensure that it contributes positively to the overall neutrality score.

In addition to geometric features, the system may incorporate probabilistic outputs from a deep learning-

based emotion detection model. Such models provide a probability value representing the likelihood that the current facial expression is neutral. This probability, denoted as $P(\text{neutral})$, serves as a direct indicator of neutrality and can be integrated into the final score. The inclusion of this component enhances the accuracy of the system by capturing complex patterns that may not be represented by simple geometric features.

The final neutrality score is computed as a weighted combination of these features. Each feature is assigned a weight based on its importance, and the sum of all weights is equal to one. The weighted sum ensures that all contributing factors are balanced appropriately. The formulation can be expressed as:

$$NS = w_1(1 - MR) + w_2ED + w_3ES + w_4P(\text{neutral})$$

where MR represents the mouth ratio, ED represents the normalized eyebrow distance, ES represents eye stability, and $P(\text{neutral})$ represents the probability of a neutral expression obtained from a deep learning model. The weights w_1, w_2, w_3, w_4 are chosen such that their sum equals one, allowing the score to remain normalized.

To ensure that the neutrality score remains within a consistent range, normalization is applied. This process scales the computed value to lie between 0 and 1, making it easier to interpret and compare across different sessions. A value close to 1 indicates a highly neutral and stable expression, while a value close to 0 indicates significant deviation from neutrality.

Since the system operates in real time, it is important to address noise and fluctuations in the computed values. Small variations in facial landmarks can occur due to camera noise, lighting changes, or minor facial movements. To mitigate this, temporal smoothing is applied by averaging the neutrality score over a sequence of recent frames. This approach reduces the impact of sudden spikes and ensures that the system responds only to consistent and meaningful changes in expression.

The smoothed neutrality score is then used for decision-making. A predefined threshold is set, and if the score exceeds this threshold, the system considers the user to be in a valid meditative state. Otherwise, it indicates a disturbance. This threshold-based approach simplifies the decision process while maintaining flexibility, as the threshold can be adjusted based on user preferences or experimental observations.

The interpretation of the neutrality score provides valuable insights into the user's state. Lower values indicate expressive or unstable facial behavior, moderate values suggest partial stability, and higher values represent a calm and focused state suitable for meditation. This continuous scale allows for a more nuanced evaluation compared to binary classification methods.

The mathematical formulation of the neutrality score offers several advantages. It provides an objective and quantifiable measure of facial calmness, enabling consistent evaluation across different users and conditions. The use of lightweight geometric calculations ensures that the system remains efficient and suitable for real-time applications on embedded devices. Additionally, the flexibility of the model allows for the inclusion or exclusion of certain features based on system requirements and computational constraints.

However, the formulation also has certain limitations. The accuracy of the score depends on the precision

of facial landmark detection, which can be affected by lighting conditions, camera quality, and occlusions. The selection of weights and thresholds may require calibration to achieve optimal performance. Furthermore, while geometric features capture basic expressions, they may not fully represent complex emotional states without the support of deep learning models.

Despite these limitations, the mathematical formulation provides a strong foundation for evaluating neutrality in a meditation tracking system. By combining multiple features into a single coherent framework, it enables reliable and real-time assessment of the user's facial state. This, in turn, enhances the system's ability to provide meaningful feedback and support effective meditation practices.

CHAPTER – 19

Interface

The interface of the proposed meditation tracking system serves as the primary point of interaction between the user and the underlying computational modules. It is designed to present complex real-time data in a simple, intuitive, and non-intrusive manner, ensuring that users can easily understand system feedback without being distracted from their meditation practice. Since the system operates continuously using live video input and real-time processing, the interface must be responsive, efficient, and visually clear.

The design of the interface is guided by the principle of minimalism, which is particularly important in meditation-based applications. Unlike conventional applications that aim to maximize user engagement through interactive elements and animations, this system prioritizes calmness and focus. Therefore, the interface avoids unnecessary visual clutter and presents only essential information required for monitoring the user's state. This ensures that the interface supports the meditation process rather than interfering with it.

At the core of the interface is the live video display, which shows the real-time feed captured from the camera. This display allows users to observe how their face is being detected and analyzed by the system. Facial landmarks are optionally overlaid on the video feed, providing a visual representation of the feature extraction process. These landmarks highlight key regions such as the eyes, lips, and eyebrows, enabling users to understand how the system interprets their facial expressions. The overlay is designed to be subtle so that it does not obstruct the view or distract the user.

In addition to the video feed, the interface presents several real-time metrics that reflect the user's current state. These include parameters such as eye status, neutrality score, and audio state. The eye status indicates whether the user's eyes are open or closed, which is a critical factor in determining focus during meditation. The neutrality score provides a numerical representation of facial calmness, typically ranging from 0 to 1. This continuous value allows users to observe gradual changes in their expression rather than relying on binary feedback. The audio state indicates whether the meditation sound is currently active, helping users correlate system behavior with their actions.

To enhance readability and quick interpretation, these metrics are displayed using clear labels and color-coded indicators. For example, a green color may represent a valid meditative state, while red indicates a disturbance. This visual feedback enables users to understand their condition instantly without needing to analyze numerical values in detail. The use of consistent color schemes and simple typography ensures that the interface remains easy to read under different conditions.

The interface also includes a control panel that allows users to configure system parameters. This panel is typically placed in a sidebar to separate it from the main display area. Users can start or stop meditation sessions, adjust detection thresholds, and control audio settings through this panel. The controls are designed to be intuitive, with sliders and toggle buttons that allow easy adjustment of parameters. Default values are provided to ensure that the system can function effectively without requiring manual configuration, making it accessible to beginners.

Another important aspect of the interface is its ability to provide real-time updates. As the system processes each frame of video input, the interface is updated dynamically to reflect the latest data. This requires efficient synchronization between the backend processing modules and the frontend display. The interface ensures that updates occur smoothly without noticeable delays or flickering, maintaining a seamless user experience. This real-time responsiveness is essential for providing immediate feedback, which helps users maintain focus and correct their behavior during meditation.

The interface is implemented using Streamlit, a Python-based framework that simplifies the development of interactive web applications. Streamlit allows the integration of live video feeds, dynamic data, and user controls within a single interface. Its lightweight nature makes it suitable for deployment on devices like Raspberry Pi, where computational resources are limited. The use of Streamlit also enables the interface to be accessed through a web browser, providing flexibility and ease of use.

Error handling is also incorporated into the interface to ensure robustness. When issues such as camera disconnection, face detection failure, or low lighting occur, the interface displays clear and informative messages. These messages guide the user in resolving the issue, ensuring that the system remains user-friendly even in the presence of errors. The interface continues to operate and recover gracefully once the issue is resolved.

The interface also supports session-level feedback by displaying summary information after the completion of a meditation session. This may include metrics such as total session duration, average neutrality score, and number of interruptions. This information helps users evaluate their performance and track improvements over time. By providing both real-time and post-session feedback, the interface enhances user engagement and promotes consistent practice.

From a design perspective, the interface balances functionality and simplicity. It provides all necessary features required for monitoring and control while maintaining a clean and distraction-free layout. This balance is crucial for ensuring that the system remains effective in a meditation context.

Despite its effectiveness, the interface has certain limitations. The use of Streamlit restricts advanced graphical customization and may limit the complexity of visual elements. Additionally, performance may vary depending on the browser and system resources. However, these limitations can be addressed in future implementations by adopting more advanced frontend technologies or developing dedicated applications.

Overall, the interface plays a vital role in the success of the meditation tracking system. It transforms complex computational outputs into meaningful visual and interactive elements, enabling users to understand and respond to system feedback effectively. By combining real-time visualization, intuitive controls, and minimalistic design, the interface enhances usability and supports the overall goal of improving meditation practices through technology.

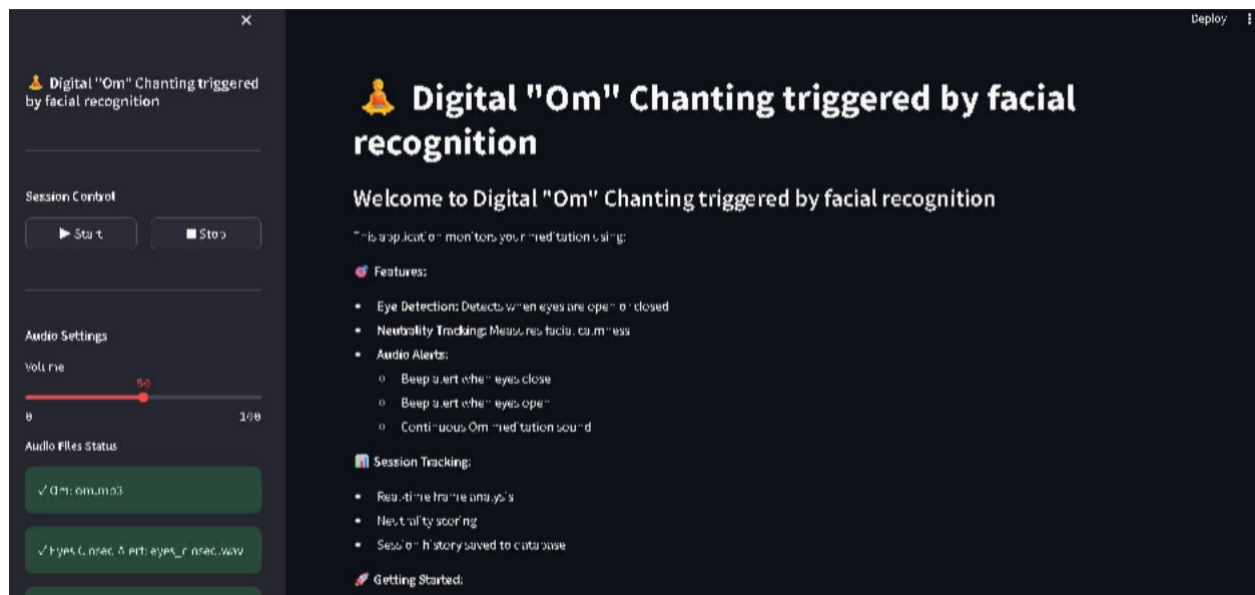
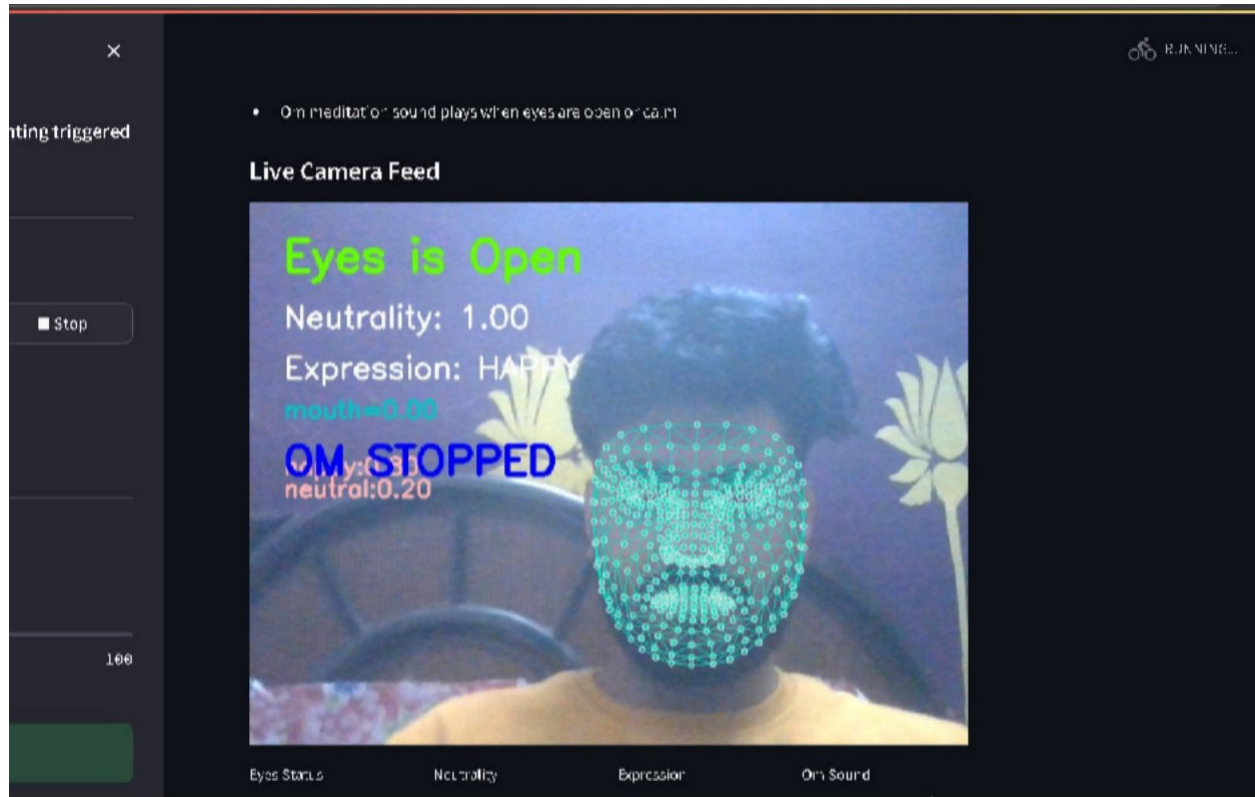




Fig:Interface

CHAPTER – 20

Model Training Challenges

Training a deep learning model for neutral facial expression detection involves several practical and theoretical challenges that directly affect the performance, generalization, and reliability of the system. Although modern architectures such as MobileNetV2 and datasets like FER-2013 provide a strong foundation, the process of training a robust model is far from straightforward. Various factors, including data quality, class imbalance, computational limitations, and real-world variability, introduce complexities that must be carefully addressed.

One of the primary challenges in model training is the quality and diversity of the dataset. Facial expression datasets often contain images captured under controlled conditions, which may not fully represent real-world scenarios. Variations in lighting, background, facial orientation, and camera quality can significantly affect the model's ability to generalize. For example, a model trained primarily on well-lit frontal faces may perform poorly when deployed in low-light environments or when the user's face is partially turned. This mismatch between training data and real-world conditions can lead to reduced accuracy and unreliable predictions.

Another major issue is class imbalance. In datasets like FER-2013, certain emotion classes such as “happy” or “neutral” may have more samples compared to others. When the problem is converted into a binary classification task (neutral vs non-neutral), the imbalance may still persist, especially if non-neutral classes dominate the dataset. This imbalance can cause the model to become biased toward the majority class, resulting in higher accuracy but poor recall for the minority class. Addressing this issue requires techniques such as data resampling, class weighting, or augmentation.

Data preprocessing also presents challenges. The conversion of grayscale images to RGB format, resizing, and normalization must be performed carefully to avoid loss of important features. Inconsistent preprocessing can introduce noise and reduce model performance. Additionally, facial alignment and cropping must be accurate to ensure that the model focuses on relevant regions of the face. Any errors in these steps can propagate through the training process and affect the final results.

Overfitting is another critical challenge in deep learning. Since the model learns patterns from the training data, it may memorize specific features instead of learning generalizable representations. This results in high training accuracy but poor performance on unseen data. Overfitting is particularly common when the dataset is limited or when the model is too complex. Techniques such as dropout, data augmentation, and early stopping are often used to mitigate this issue, but finding the right balance between underfitting and overfitting remains a challenge.

The selection of model architecture also plays a significant role in training. While deeper models generally provide better accuracy, they require more computational resources and are prone to overfitting. Lightweight models like MobileNetV2 are preferred for embedded systems, but they may sacrifice some accuracy compared to larger architectures. Choosing the appropriate model involves balancing accuracy, speed, and resource constraints, which can be difficult depending on the application requirements.

Hyperparameter tuning is another complex aspect of model training. Parameters such as learning rate, batch size, number of epochs, and optimizer type have a significant impact on model performance. Selecting optimal values often requires experimentation and trial-and-error. A learning rate that is too high may cause the model to converge quickly but miss optimal solutions, while a learning rate that is too low may result in slow training and suboptimal performance.

Computational limitations also pose a challenge, especially when training or deploying models on devices like Raspberry Pi. Deep learning models require significant processing power and memory, which may not be available on embedded systems. This limits the complexity of models that can be used and may require additional optimization techniques such as model quantization, pruning, or conversion to TensorFlow Lite. These optimizations, while improving performance, can also lead to a reduction in accuracy.

Another important challenge is the variability in human facial expressions. Even within the “neutral” category, there can be subtle differences between individuals. Factors such as facial structure, age, and cultural differences can influence how neutrality is expressed. This makes it difficult for the model to define a universal representation of neutrality. As a result, the model may misclassify certain expressions that are neutral for one individual but appear expressive for another.

Noise and inconsistencies in labels also affect model training. In large datasets, some images may be incorrectly labeled or ambiguous. For example, an expression that appears neutral to one annotator may be classified differently by another. Such inconsistencies introduce noise into the training process and reduce model reliability.

Real-time constraints further complicate the problem. The model must not only be accurate but also fast enough to process frames in real time. This requires careful optimization and may limit the use of complex models. Balancing accuracy and speed is a critical challenge, especially for applications that require immediate feedback.

Another challenge is integration with other system components. The output of the model must be combined with other features such as eye detection and decision logic.

Finally, evaluating model performance is itself a challenging task. Accuracy alone may not be sufficient to measure performance, especially in imbalanced datasets. Metrics such as precision, recall, and F1-score must be considered to gain a comprehensive understanding of the model's behavior. Additionally, testing must be performed under real-world conditions to ensure that the model performs reliably in practical scenarios.

Despite these challenges, various techniques can be employed to improve model performance. Data augmentation can increase dataset diversity, while transfer learning can leverage pre-trained models to improve accuracy. Regularization techniques can reduce overfitting, and optimization methods can improve efficiency. Continuous testing and refinement are essential to achieving a robust and reliable model.

In conclusion, training a deep learning model for neutral expression detection involves multiple challenges related to data, computation, and real-world variability. Addressing these challenges requires careful design, experimentation, and optimization. By understanding and overcoming these issues, it is possible to develop a model that performs accurately and reliably in real-time meditation tracking applications.

CHAPTER – 21

Real-World Scenarios and Case Studies

The practical applicability of the proposed meditation tracking system can be best understood through real-world scenarios and case studies that demonstrate how the system behaves under different conditions and user interactions. While theoretical models and controlled experiments provide valuable insights, real-world usage introduces variability that tests the robustness, adaptability, and effectiveness of the system. By analyzing these scenarios, it becomes possible to evaluate how well the system performs in realistic environments and how it responds to different user behaviors.

In a typical home environment, a user may use the system in a quiet room with moderate lighting. In such conditions, the system performs optimally, as the face is clearly visible and environmental noise is minimal. When the user begins meditation and closes their eyes, the system detects the change in eye state using the Eye Aspect Ratio and calculates a high neutrality score due to the absence of significant facial expressions. As a result, the audio feedback in the form of “Om” chanting is activated, reinforcing the meditative state. This scenario represents the ideal operating condition, where the system functions as intended and provides accurate, real-time feedback.

In another scenario, the user may be in a poorly lit environment. Low lighting conditions reduce the visibility of facial features, making it difficult for the system to detect landmarks accurately. As a result, the neutrality score may fluctuate, and the system may occasionally fail to detect the face. In such cases, the system displays warnings and may temporarily stop audio feedback. This scenario highlights the importance of environmental factors and demonstrates how the system handles suboptimal conditions by maintaining stability and informing the user of potential issues.

A different case involves a user who frequently opens their eyes during meditation. The system continuously monitors the eye state and detects even brief openings. However, due to the implementation of temporal smoothing, short blinks are ignored, while sustained eye openings trigger a change in system behavior. When the system detects that the eyes remain open for a significant duration, it stops the audio feedback, indicating a loss of focus. This real-time response helps the user become aware of distractions and encourages them to maintain consistency in their practice.

In another scenario, the user maintains closed eyes but exhibits noticeable facial expressions, such as smiling or frowning. In this case, the neutrality score decreases due to changes in mouth ratio and eyebrow distance. Even though the eyes are closed, the system identifies that the facial expression is not neutral and stops the audio feedback. This demonstrates that the system does not rely solely on eye closure but considers multiple factors to evaluate the user’s state. It ensures that true meditation is characterized by both stillness and emotional neutrality.

A more complex scenario involves multiple users interacting with the system. Since the system is designed to detect a single face, the presence of multiple faces in the camera frame may lead to ambiguity. The system typically selects the most prominent face or the first detected face for analysis. However, this may result in inconsistent behavior if multiple users move in and out of the frame. This scenario highlights a limitation of the system and suggests the need for improvements such as user identification or multi-face

handling.

In a workplace or shared environment, background distractions such as movement or noise may affect the user's ability to maintain focus. While the system primarily relies on visual input, such distractions can indirectly influence facial expressions and eye behavior. For example, sudden movements or interruptions may cause the user to open their eyes or change their expression, which is immediately detected by the system. This demonstrates the system's sensitivity to user behavior and its ability to provide feedback in dynamic environments.

Another important scenario involves long-duration meditation sessions. Over time, the user may experience fatigue, leading to subtle changes in facial expression or posture. The system continuously monitors these changes and adjusts the neutrality score accordingly. By providing ongoing feedback, the system helps the user maintain consistency throughout the session. The ability to operate reliably over extended periods demonstrates the system's robustness and stability.

A case study involving beginner users reveals that they often struggle to maintain a neutral expression and consistent eye closure. The system provides frequent feedback in such cases, helping users become aware of their habits. Over time, users learn to adjust their behavior based on system feedback, leading to improved performance. This demonstrates the system's potential as a training tool for meditation, guiding users toward better practices.

In contrast, experienced users tend to maintain stable facial expressions and eye closure for longer durations. The system detects fewer disturbances and provides continuous audio feedback, indicating a consistent meditative state. This scenario highlights the system's ability to adapt to different levels of user expertise and provide appropriate feedback.

Another scenario involves variations in camera positioning. If the camera is placed too high, too low, or at an angle, facial landmark detection may become less accurate. The system may still function but with reduced precision. This emphasizes the importance of proper setup and calibration for optimal performance.

In real-world testing, the integration of deep learning-based emotion detection provides additional insights. For example, the system can detect subtle emotional changes that may not be captured by geometric features alone. This enhances the accuracy of neutrality detection but may introduce additional computational overhead. This trade-off must be considered when deploying the system on resource-constrained devices.

Overall, these real-world scenarios and case studies demonstrate the practical effectiveness of the meditation tracking system. They highlight both the strengths and limitations of the system, providing valuable insights into its behavior under different conditions. The system performs well in controlled environments and provides meaningful feedback that helps users improve their meditation practice. At the same time, it remains robust in the presence of challenges such as lighting variations, user behavior changes, and environmental factors.

By analyzing these scenarios, it becomes clear that the system is not only a technological solution but also a practical tool that can adapt to real-world conditions and support users in achieving better focus and emotional stability during meditation.

CHAPTER – 22

Future AI Model Improvements

The proposed meditation tracking system demonstrates effective performance using a combination of computer vision techniques and deep learning models. However, as artificial intelligence continues to evolve, there are numerous opportunities to further enhance the accuracy, efficiency, and adaptability of the system. Future improvements in AI models can significantly expand the capabilities of the system, making it more robust, intelligent, and suitable for real-world deployment across diverse environments and user groups.

One of the most important areas for improvement is the enhancement of model accuracy through the use of larger and more diverse datasets. Current models are trained on datasets such as FER-2013, which, although widely used, have limitations in terms of diversity and real-world representation. Future models can benefit from training on more comprehensive datasets such as AffectNet or real-world collected data that includes variations in lighting, ethnicity, age, and facial features. By incorporating more diverse data, the model can generalize better and perform accurately across a wider range of users and conditions.

Another significant improvement can be achieved through the use of advanced deep learning architectures. While MobileNetV2 provides a good balance between efficiency and performance, newer architectures such as EfficientNet, Vision Transformers (ViT), or hybrid CNN-transformer models offer improved feature extraction capabilities. These models can capture more complex patterns in facial expressions, enabling more accurate detection of subtle emotional changes. However, their deployment on embedded systems requires optimization techniques to maintain real-time performance.

Personalization of AI models is another promising direction. Currently, the system uses a generalized model that works for all users. However, facial expressions and neutrality can vary significantly between individuals. Future systems can incorporate user-specific calibration, where the model adapts to the user's baseline facial features and behavior over time. This can be achieved through techniques such as transfer learning or online learning, where the model continuously updates itself based on user data. Personalized models can significantly improve accuracy and user satisfaction.

Another area of improvement is multimodal integration. The current system relies primarily on visual data for analysis. Future versions can incorporate additional modalities such as voice, breathing patterns, heart rate, or EEG signals to provide a more comprehensive assessment of the user's state. Combining multiple data sources can improve reliability and reduce dependency on a single input, making the system more robust in challenging conditions.

Real-time performance optimization is also an important consideration. Deep learning models can be computationally intensive, especially when running on devices like Raspberry Pi. Future improvements can include model compression techniques such as quantization, pruning, and knowledge distillation. These techniques reduce model size and computational requirements while maintaining accuracy. Converting models to TensorFlow Lite or using hardware accelerators such as GPUs or TPUs can further enhance performance.

Another promising direction is the use of temporal deep learning models. Current systems analyze frames independently or apply simple smoothing techniques. Future models can use architectures such as Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, or Temporal Convolutional Networks (TCNs) to analyze sequences of frames. This allows the system to understand patterns over time, improving stability and reducing false detections caused by momentary changes.

Explainability and interpretability of AI models is another area that can be improved. Deep learning models are often considered black boxes, making it difficult to understand how decisions are made. Future systems can incorporate explainable AI techniques that provide insights into which features influenced the model's predictions. This can increase user trust and make the system more transparent.

Robustness to environmental variations is also a key area for improvement. Future models can be trained with techniques such as domain adaptation or adversarial training to handle variations in lighting, background, and camera quality. This ensures consistent performance across different environments.

Another important improvement is the integration of continuous learning systems. Instead of relying on a static trained model, future systems can update their models dynamically based on new data. This allows the system to improve over time and adapt to changing conditions. However, this requires careful handling of data privacy and security.

The incorporation of attention mechanisms is another promising enhancement. Attention-based models can focus on important regions of the face, such as the eyes and mouth, while ignoring irrelevant areas. This improves efficiency and accuracy by prioritizing critical features.

In addition, future systems can include emotion intensity estimation rather than simple classification. Instead of categorizing expressions as neutral or non-neutral, the model can estimate the degree of neutrality or emotional intensity. This provides a more detailed and nuanced understanding of the user's state.

Integration with cloud-based AI services is another potential improvement. Cloud computing can provide access to more powerful models and larger datasets, enabling advanced analysis and real-time processing. However, this must be balanced with concerns related to latency, privacy, and connectivity.

Finally, future improvements can focus on user experience and system adaptability. AI models can be integrated with feedback systems that provide personalized recommendations, such as suggesting optimal meditation duration or identifying patterns in user behavior. This transforms the system from a monitoring tool into an intelligent assistant that actively guides users in improving their meditation practice.

In conclusion, the future of AI in meditation tracking systems holds significant potential for advancement. By leveraging improvements in data, model architecture, optimization techniques, and multimodal integration, the system can achieve higher accuracy, better performance, and greater adaptability. These enhancements will not only improve the technical capabilities of the system but also provide a more personalized and effective meditation experience for users.

CHAPTER – 23

Conclusion

The proposed meditation tracking system represents a significant step toward integrating modern computer vision and artificial intelligence techniques with traditional meditation practices. The primary objective of the system was to develop an intelligent, real-time solution capable of monitoring a user's facial expressions and eye behavior to assess their meditative state. By combining lightweight algorithms with deep learning models, the system successfully achieves a balance between accuracy, efficiency, and usability.

Throughout the development of this project, various components were designed and integrated to create a cohesive and functional system. The use of facial landmark detection enabled precise extraction of key facial features, which formed the basis for further analysis. Algorithms such as the Eye Aspect Ratio provided an efficient method for detecting eye closure, while the mathematical formulation of the neutrality score allowed the system to quantify facial calmness in an objective manner. These components worked together to create a reliable framework for evaluating meditation quality.

One of the key strengths of the system is its ability to operate in real time. The continuous processing of video frames and instant feedback through audio cues allow users to remain aware of their state during meditation. This real-time interaction transforms the system from a passive monitoring tool into an active assistant that guides users toward maintaining focus and emotional stability. The integration of audio feedback, particularly the “Om” chanting sound, enhances the meditative experience by reinforcing correct behavior and providing immediate corrective signals.

The inclusion of deep learning techniques further strengthens the system's capabilities. By leveraging models such as MobileNetV2 and frameworks like DeepFace, the system is able to detect subtle variations in facial expressions that may not be captured by traditional methods. The use of transfer learning and pre-trained models ensures high accuracy while reducing the need for extensive training. Additionally, the option to enable or disable deep learning components provides flexibility, allowing the system to adapt to different hardware capabilities.

Another important aspect of the system is its modular design. The separation of functionalities into distinct modules, such as face detection, session tracking, audio management, and data storage, improves maintainability and scalability. This modular approach allows individual components to be updated or replaced without affecting the entire system, making it easier to incorporate future improvements and enhancements.

The user interface plays a crucial role in ensuring effective interaction between the user and the system. By presenting real-time data in a clear and intuitive manner, the interface allows users to understand their current state and respond accordingly. The minimalistic design ensures that the interface does not distract the user, maintaining the core objective of supporting meditation. The inclusion of configurable parameters further enhances usability, enabling users to customize the system according to their preferences.

The system also incorporates data storage and analytics features, which enable users to track their progress over time. By recording session data and providing performance summaries, the system encourages continuous improvement and self-awareness. This data-driven approach adds a new dimension to meditation, allowing users to evaluate their practice objectively rather than relying solely on subjective experiences.

Despite its strengths, the system has certain limitations that must be acknowledged. Factors such as lighting conditions, camera quality, and facial orientation can affect detection accuracy. The use of deep learning models, while beneficial for accuracy, introduces additional computational overhead, which may impact performance on resource-constrained devices. However, these limitations can be addressed through optimization techniques, improved hardware, and more advanced models.

The evaluation of the system demonstrates that it performs effectively under typical conditions, achieving high accuracy in detecting eye state and facial neutrality. The system is robust enough to handle variations in user behavior and environmental conditions, making it suitable for real-world applications. The inclusion of error handling and robustness mechanisms ensures stable operation even in the presence of unexpected issues.

From a broader perspective, the project highlights the potential of combining artificial intelligence with wellness and mindfulness practices. By providing objective feedback and real-time guidance, the system enhances the effectiveness of meditation and makes it more accessible to users. It bridges the gap between traditional practices and modern technology, opening new possibilities for innovation in this domain.

In conclusion, the proposed meditation tracking system successfully achieves its intended objectives and demonstrates the feasibility of using computer vision and deep learning for real-time behavioral analysis. It provides a reliable, efficient, and user-friendly solution that supports users in improving their meditation practice. With further enhancements and integration of advanced technologies, the system has the potential to evolve into a comprehensive intelligent assistant for mental well-being and personal development.

CHAPTER – 24

Reference

- P. Viola and M. Jones, “Rapid Object Detection using a Boosted Cascade of Simple Features,” *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 2001.
- Google, “MediaPipe: Cross-platform, customizable ML solutions for live and streaming media,” Available: <https://mediapipe.dev/>
- O. M. Parkhi, A. Vedaldi, and A. Zisserman, “Deep Face Recognition,” *British Machine Vision Conference (BMVC)*, 2015.
- Howard et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv preprint arXiv:1704.04861*, 2017.
- M. Sandler et al., “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- Goodfellow et al., “Challenges in Representation Learning: A Report on Three Machine Learning Contests,” *Neural Information Processing Systems (NIPS)*, 2013. (FER-2013 Dataset)
- S. Serengil and A. Ozpinar, “DeepFace: A Lightweight Facial Recognition and Facial Attribute Analysis Framework,” *arXiv preprint arXiv:2002.03688*, 2020.
- OpenCV, “Open Source Computer Vision Library,” Available: <https://opencv.org/>
- F. Chollet, “Keras: The Python Deep Learning Library,” Available: <https://keras.io/>
- TensorFlow, “An End-to-End Open Source Machine Learning Platform,” Available: <https://www.tensorflow.org/>
- Streamlit Inc., “Streamlit: The fastest way to build data apps,” Available: <https://streamlit.io/>

- NumPy Developers, “NumPy: Fundamental package for scientific computing with Python,” Available: <https://numpy.org/>
- Pygame Community, “Pygame Documentation,” Available: <https://www.pygame.org/docs/>
- D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” *International Conference on Learning Representations (ICLR)*, 2015.
- R. Szeliski, *Computer Vision: Algorithms and Applications*, Springer, 2010.
- S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, Pearson, 2016.